**REPORTE DE PROYECTO DE ESTADÍA**

# DATACORE: Plataforma Inteligente de Gestión, Monitoreo y Análisis de Datos Empresariales.

**PROGRAMA EDUCATIVO**

TSU en Datos e Inteligencia Artificial

**PRESENTA**

David Israel Atilano Quiroz
Ricardo Leonel Ordaz Aguilar

**DIRECTOR UPQ:**

René Santana Cruz

**CO DIRECTOR UPQ:**

Adela Becerra Chávez

**EMPRESA:**

Universidad politécnica de Querétaro.

**ASESOR EXTERNO:**

Dra. Karina Anaya

El Marqués, Querétaro. Noviembre 2026

## Resumen.

El presente reporte documenta el proyecto de estadía DATACORE: Plataforma Inteligente de Gestión, Monitoreo y Análisis de Datos Empresariales, desarrollado para NexaTech Solutions S.A. de C.V., una distribuidora ficticia de productos de consumo con 12 sucursales en 9 estados de la República Mexicana.

Para probar la plataforma se simularon tres años de operación (2023 a 2025): 181,054 tickets de venta, 640,673 renglones de detalle y 647,613 movimientos de inventario. Se inyectaron 16,502 defectos de 18 tipos; el ETL los detecta todos con 100 % de precisión y entre 97.3 % y 100 % de exhaustividad, y carga los datos en PostgreSQL en unos 50 segundos.

El módulo de inteligencia artificial pronostica la demanda semanal de los 4,800 pares sucursal-producto con un modelo de gradient boosting con pérdida Poisson. Validado con origen móvil sobre las 52 semanas de 2025, alcanza un WAPE por producto y semana de 26.9 %, frente a 28.8 % de la media móvil, el mejor de cuatro métodos de referencia, y supera a todos ellos en los 12 casos evaluados. Con el pronóstico se generan alertas de riesgo de desabasto, exceso de inventario y productos sin rotación, cada una con su explicación y, en el caso del desabasto, con una cantidad y una fecha límite de pedido.

Una simulación de 2025 sobre la demanda real muestra que reponer con DATACORE habría reducido 14 % las ventas perdidas y 12 % el inventario inmovilizado, con un beneficio neto estimado de 346 mil pesos al año. La plataforma se expone en una API REST y un dashboard, con respaldos verificados, monitoreo, un proceso diario, un contenedor Docker e integración continua con 116 pruebas automatizadas.



**Palabras clave:** ingeniería de datos, ETL, calidad de datos, pronóstico de demanda, gestión de inventarios.

## Abstract.

This report documents the internship project DATACORE: an Intelligent Platform for the Management, Monitoring and Analysis of Business Data, developed for NexaTech Solutions S.A. de C.V., a fictitious consumer-goods distributor with 12 branches in 9 Mexican states. The platform spans from the files exported by each business area to the inventory alerts the user reviews.

Three years of operations (2023 to 2025) were simulated to test the platform: 181,054 sales tickets, 640,673 sales lines and 647,613 inventory movements. While exporting the data, 16,502 defects of 18 types were deliberately injected and logged as ground truth. The ETL process applies data-quality rules that detect all 18 defect types with 100 % precision and 97.3 % to 100 % recall, and it loads the validated data into PostgreSQL in about 50 seconds.

The artificial-intelligence module forecasts weekly demand for the 4,800 branch-product pairs with a gradient boosting model trained with a Poisson loss. Validated with a rolling origin over the 52 weeks of 2025, it reaches a product-week WAPE of 26.9 %, compared with 28.8 % for the moving average, the best of four baseline methods, and it outperforms all of them in the 12 evaluated cases. The forecast feeds alerts for stock-out risk, excess inventory and non-moving products, each with its explanation and, for stock-outs, a suggested order quantity and deadline.

A simulation of 2025 on actual demand shows that replenishing with DATACORE would have cut lost sales by 14 % and inventory by 12 %, for an estimated net benefit of 346 thousand pesos a year. The platform is served through a REST API and a dashboard, with verified backups, monitoring, a daily process, a Docker container, and continuous integration with 116 automated tests.



**Keywords:** data engineering, ETL, data quality, demand forecasting, inventory management.

# ÍNDICE

Resumen....2
Abstract....3
ÍNDICE....4
Índice de figuras....7
Índice de tablas....8
CAPÍTULO I ANTECEDENTES....10
1.1 Contexto del proyecto de estadía....11
1.2 La empresa NexaTech Solutions....11
1.3 Sistemas de información y fuentes de datos....12
1.4 Situación de la gestión de datos y del inventario....13
1.5 Antecedentes tecnológicos....14
CAPÍTULO II MARCO TEÓRICO....15
2.1 Ingeniería de datos y arquitectura por capas....16
2.2 Procesos ETL....16
2.3 Calidad de datos....17
2.4 Modelo relacional y PostgreSQL....17
2.5 Análisis exploratorio de datos....18
2.6 Pronóstico de demanda....19
2.6.1 Series de tiempo y demanda intermitente....19
2.6.2 Gradient boosting con pérdida Poisson....19
2.6.3 Validación temporal y fuga de información....20
2.6.4 Métricas de error....20
2.6.5 Demanda censurada....20
2.6.6 Intervalos de pronóstico....21
2.7 Gestión de inventarios....21
2.8 Detección de anomalías....21
2.9 Servicios web y visualización....22
2.10 Operación de plataformas de datos....22
2.11 Objetivos de Desarrollo Sostenible....23
CAPÍTULO III DEFINICIÓN DEL PROYECTO....24
3.1 Planteamiento del problema....25
3.2 Justificación....25

3.3 Alcances y restricciones....26
3.4 Objetivo General....27
3.5 Objetivos específicos....27
CAPÍTULO IV DESARROLLO DEL PROYECTO....29
4.1 Desarrollo de las etapas....30
4.1.1 Metodología y organización del trabajo....30
4.1.2 Primera etapa: requerimientos y alcance....31
4.1.3 Segunda etapa: arquitectura y diseño de la base de datos....32
4.1.4 Tercera etapa: simulación de datos....36
4.1.5 Cuarta etapa: ETL y reglas de calidad....38
4.1.6 Quinta etapa: análisis exploratorio e indicadores....40
4.1.7 Sexta etapa: modelo de pronóstico de demanda....45
4.1.8 Séptima etapa: API y dashboard....48
4.1.9 Octava etapa: operación, respaldos y monitoreo....55
4.1.10 Pruebas del sistema....57
4.1.11 Novena etapa: de plataforma que funciona a plataforma que demuestra su valor....58
4.1.12 Mejora 1: impacto en dinero....58
4.1.13 Mejora 2: alertas de desabasto más precisas....59
4.1.14 Mejora 3: de alertar a recomendar....60
4.1.15 Mejora 4: explicar cada alerta, rediseño y preparación del despliegue....61
4.2 Análisis de los resultados....62
4.2.1 Resumen de resultados....62
4.2.2 Calidad de los datos....63
4.2.3 Pronóstico de demanda....64
4.2.4 Alertas de desabasto (regla original)....66
4.2.5 Detección de anomalías....67
4.2.6 Operación de la plataforma....68
4.2.7 Cumplimiento de los requerimientos....69
4.2.8 Impacto económico de DATACORE....69
4.2.9 Alertas de desabasto mejoradas....71
CAPÍTULO V CONCLUSIONES....73
5.1 Conclusiones generales....74
5.2 Cumplimiento de los objetivos....75
5.3 Relación con las competencias de la estadía....75
5.4 Limitaciones....75

5.5 Trabajo futuro....76
5.6 Experiencia del equipo....77
5.6.1 Reflexión de David Israel Atilano Quiroz....77
5.6.2 Reflexión de Ricardo Leonel Ordaz Aguilar....78
REFERENCIAS BIBLIOGRÁFICAS....80
ANEXOS....82
Anexo 1. Diccionario de datos....82
Anexo 2. Guía rápida de operación....84
Anexo 3. Guía rápida para el usuario del dashboard....85

# Índice de figuras

Figura 1.1. Logotipo de DATACORE....11
Figura 4.1. Cronograma del proyecto en fases de dos semanas....30
Figura 4.2. Arquitectura por capas de DATACORE....33
Figura 4.3. Modelo entidad-relación del esquema core....35
Figura 4.4. Defectos inyectados en los archivos crudos por tipo....38
Figura 4.5. Flujo del proceso ETL....38
Figura 4.6. Ingreso mensual de NexaTech Solutions, 2023 a 2025....41
Figura 4.7. Índice estacional por categoría y mes....41
Figura 4.8. Ingreso y ticket promedio por sucursal....42
Figura 4.9. Curva de Pareto del ingreso por producto....42
Figura 4.10. Ingreso por día de la semana y por hora....43
Figura 4.11. Participación en el ingreso por método de pago y segmento de cliente....43
Figura 4.12. Distribución de la cobertura de inventario al cierre de 2025....44
Figura 4.13. Incidencias mensuales por tipo....44
Figura 4.14. Página de Inicio del dashboard....50
Figura 4.15. Página de Ventas del dashboard....51
Figura 4.16. Página de Inventario del dashboard....52
Figura 4.17. Página de Alertas del dashboard....53
Figura 4.18. Página de Pronóstico del dashboard....54
Figura 4.19. Página de Calidad y operación del dashboard....55
Figura 4.20. Pasos del proceso diario automatizado....56
Figura 4.21. WAPE del modelo contra los métodos de referencia....65
Figura 4.22. Precisión y exhaustividad de las reglas de alerta de desabasto....67
Figura 4.23. Exhaustividad del detector según la duración de la falla....68
Figura 4.24. Ventas perdidas contra inventario inmovilizado por política, 2025....71
Figura 4.25. F1 por trimestre de 2025 de las reglas de alerta de desabasto....72

# Índice de tablas

Tabla 1.1. Sucursales de NexaTech Solutions....12
Tabla 1.2. Fuentes de datos por área de NexaTech Solutions....13
Tabla 2.1. Dimensiones de la calidad de datos....17
Tabla 3.1. Alcance del proyecto....26
Tabla 4.1. Requerimientos funcionales....31
Tabla 4.2. Requerimientos no funcionales....32
Tabla 4.3. Riesgos del proyecto y su mitigación....32
Tabla 4.4. Tecnologías por capa....33
Tabla 4.5. Decisiones de diseño....34
Tabla 4.6. Esquemas de la base de datos....35
Tabla 4.7. Volumen de datos generados con la semilla 42....36
Tabla 4.8. Reglas de generación de la demanda....37
Tabla 4.9. Reglas de calidad principales del ETL....39
Tabla 4.10. Vistas de indicadores del esquema kpi....45
Tabla 4.11. Decisiones de diseño del modelo de demanda....45
Tabla 4.12. Variables del modelo de demanda....46
Tabla 4.13. Reglas de las alertas de inventario....47
Tabla 4.14. Rutas de la API agrupadas por tema....48
Tabla 4.15. Páginas del dashboard....48
Tabla 4.16. Mecanismos de operación de la plataforma....55
Tabla 4.17. Pruebas automatizadas por módulo....57
Tabla 4.18. Resumen de resultados por componente....62
Tabla 4.19. Detección de los defectos inyectados por tipo....63
Tabla 4.20. Carga de datos por tabla....64
Tabla 4.21. WAPE por par sucursal-producto y semana según la clase ABC (horizonte de una semana)....66
Tabla 4.22. Efecto de excluir las semanas censuradas (producto-semana, horizonte de una semana)....66
Tabla 4.23. Evaluación de las reglas de alerta de desabasto....67
Tabla 4.24. Evaluación del detector de anomalías....68
Tabla 4.25. Cumplimiento de los requerimientos funcionales....69
Tabla 4.26. Impacto de DATACORE contra la política actual (2025)....70
Tabla 4.27. Reglas de alerta de desabasto evaluadas fuera de muestra (2025)....71
Tabla 5.1. Cumplimiento de los objetivos específicos....75

Tabla 5.2. Aplicación de las competencias de la estadía.....75
Tabla A.1. Diccionario de datos: core.sucursales (12 filas).....82
Tabla A.2. Diccionario de datos: core.categorias (10 filas) y core.proveedores (35 filas)...82
Tabla A.3. Diccionario de datos: core.productos (400 filas).....82
Tabla A.4. Diccionario de datos: core.clientes (8,000 filas).....83
Tabla A.5. Diccionario de datos: core.ventas (unas 181 mil filas).....83
Tabla A.6. Diccionario de datos: core.detalle_venta (unas 641 mil filas).....83
Tabla A.7. Diccionario de datos: core.inventario y core.movimientos_inventario.....83
Tabla A.8. Diccionario de datos: core.incidencias (unas 6,700 filas).....84
Tabla A.9. Tablas del esquema ops.....84
Tabla A.10. Comandos principales de la plataforma.....84
Tabla A.11. Interpretación de las alertas para el usuario.....85

# CAPÍTULO I ANTECEDENTES

## ANTECEDENTES

## 1.1 Contexto del proyecto de estadía

La estadía es el periodo en el que el estudiante del programa de Técnico Superior Universitario aplica en un proyecto completo las competencias adquiridas durante su formación. El presente proyecto se desarrolló en la Universidad Politécnica de Querétaro (UPQ), dentro del programa educativo de TSU en Datos e Inteligencia Artificial, por David Israel Atilano Quiroz y Ricardo Leonel Ordaz Aguilar, con una duración planeada de 16 semanas.

El equipo optó por un escenario empresarial simulado en lugar de una empresa real. La decisión responde a tres razones. La primera es que permite trabajar con un volumen y una variedad de datos comparables a los de una organización mediana sin depender de información confidencial ni de convenios de acceso. La segunda es que el escenario se puede controlar: se conoce de antemano qué defectos tienen los datos y cuál es la demanda real de cada producto, lo que permite medir con exactitud qué tan bien funciona cada componente. La tercera es que el proyecto no se limita a un modelo de aprendizaje automático aislado, sino que integra administración de datos, infraestructura, calidad de la información, análisis, inteligencia artificial, visualización y operación en una misma solución.

El nombre del proyecto, DATACORE, alude al papel que cumple la plataforma: ser el núcleo donde se concentran, validan y analizan los datos de la empresa. La Figura 1.1 muestra la identidad visual que se diseñó para el proyecto y que se usa en el dashboard.




**Figura 1.1.** *Logotipo de DATACORE*

*Nota.* Elaboración propia. Los colores corresponden al granate institucional de la UPQ (#8A1538) y a un azul marino (#1B2A4A).

## 1.2 La empresa NexaTech Solutions

NexaTech Solutions S.A. de C.V. es una empresa ficticia, diseñada exclusivamente con fines académicos, que se dedica a la comercialización y distribución de

productos de consumo. Opera 12 sucursales en 9 estados, agrupadas en cuatro regiones: Bajío, Centro, Occidente y Norte. Cada sucursal tiene un tamaño (grande, mediana o pequeña) que determina su afluencia de clientes. Dos de ellas, El Marqués (QRO03) y Morelia (MOR01), abrieron dentro del periodo analizado, en abril y septiembre de 2024, respectivamente. La Tabla 1.1 resume la red de sucursales.

**Tabla 1.1.** *Sucursales de NexaTech Solutions*

| Código | Nombre | Estado | Región | Tamaño | Apertura |
|---|---|---|---|---|---|
| QRO01 | Querétaro Centro | Querétaro | Bajío | Grande | marzo de 2015 |
| QRO02 | Querétaro Juriquilla | Querétaro | Bajío | Mediana | agosto de 2018 |
| QRO03 | El Marqués | Querétaro | Bajío | Pequeña | abril de 2024 |
| CDMX01 | CDMX Coyoacán | Ciudad de México | Centro | Grande | junio de 2012 |
| CDMX02 | CDMX Azcapotzalco | Ciudad de México | Centro | Mediana | febrero de 2016 |
| GDL01 | Guadalajara Zapopan | Jalisco | Occidente | Grande | septiembre de 2014 |
| MTY01 | Monterrey San Nicolás | Nuevo León | Norte | Grande | mayo de 2017 |
| LEO01 | León Centro | Guanajuato | Bajío | Mediana | enero de 2019 |
| SLP01 | San Luis Potosí | San Luis Potosí | Bajío | Pequeña | octubre de 2020 |
| PUE01 | Puebla Angelópolis | Puebla | Centro | Mediana | marzo de 2018 |
| AGS01 | Aguascalientes | Aguascalientes | Bajío | Pequeña | julio de 2021 |
| MOR01 | Morelia | Michoacán | Occidente | Pequeña | septiembre de 2024 |

*Nota.* Elaboración propia con base en la configuración del generador de datos.

El catálogo comprende 400 productos de 10 categorías: Abarrotes, Bebidas, Limpieza, Cuidado personal, Electrónica, Papelería, Juguetes, Hogar, Mascotas y Dulcería y botanas. Los productos los surten 35 proveedores, cada uno con un tiempo de entrega comprometido de 2 a 10 días. Las marcas son inventadas. La empresa atiende a unos 8,000 clientes registrados, de los cuales alrededor del 10 % son mayoristas, y además vende a público general sin registro, que representa cerca del 30 % de los tickets.

## 1.3 Sistemas de información y fuentes de datos

Como ocurre en muchas empresas medianas, cada área de NexaTech genera su información en un sistema distinto y la exporta en su propio formato. No existe un repositorio central: para responder una pregunta que cruce áreas, por ejemplo qué

productos se agotaron en las sucursales que más venden, alguien tiene que reunir y conciliar los archivos a mano. La Tabla 1.2 muestra las fuentes de datos.

**Tabla 1.2.** *Fuentes de datos por área de NexaTech Solutions*

| Área | Sistema de origen | Formato de exportación |
|---|---|---|
| Ventas | Punto de venta (POS) | CSV por año (ventas_AAAA.csv y detalle_venta_AAAA.csv) |
| Compras | Hojas de cálculo | Excel (productos.xlsx y proveedores.xlsx) |
| Mercadotecnia | CRM | CSV en Windows-1252 separado por punto y coma (clientes.csv) |
| Almacén | Sistema de inventarios | CSV (movimientos_inventario_AAAA.csv e inventario_actual.csv) |
| Operaciones | Mesa de ayuda | CSV (incidencias.csv) |

*Nota.* Elaboración propia.

Las fuentes difieren en codificación de caracteres, separadores, formatos de fecha y forma de identificar las entidades. Los archivos usan códigos de negocio, como el código de sucursal o el SKU del producto, en lugar de llaves internas. Además, los datos llegan con los problemas típicos de la captura manual: duplicados, valores faltantes, formatos inconsistentes y errores de digitación.

## 1.4 Situación de la gestión de datos y del inventario

El resurtido de cada sucursal se decide con estimaciones manuales basadas en las ventas recientes. Este método tiene dos debilidades. La primera es que no anticipa la estacionalidad: la demanda de juguetes casi se duplica en diciembre, la de papelería sube en agosto y el Buen Fin multiplica las ventas de varias categorías. La segunda es más sutil: cuando un producto se agota, la venta registrada es menor que la demanda real, y si la siguiente estimación se basa en esas ventas, el pedido vuelve a quedarse corto.

El resultado son dos problemas simultáneos. Por un lado, alrededor del 4.7 % de la demanda queda sin surtir por falta de existencia. Por otro, hay productos con existencia para más de seis meses o que no se han vendido en meses, lo que inmoviliza capital y espacio de almacén. Al cierre de 2025, 1,098 de los 4,800 pares sucursal-producto tenían cobertura superior a 180 días y 120 estaban sin existencia.

A esto se suman la falta de mecanismos automatizados para supervisar la calidad de los datos, respaldar la información y monitorear los servicios. Los procesos dependen de revisiones manuales y de información repartida en varias fuentes.

## 1.5 Antecedentes tecnológicos

Las plataformas de datos modernas suelen organizarse en capas: ingesta, almacenamiento, procesamiento, analítica y consumo (Kleppmann, 2017). Las grandes empresas resuelven estas capas con productos comerciales de almacén de datos, orquestación e inteligencia de negocios. Para una empresa mediana, la combinación de herramientas de código abierto como Python, PostgreSQL y bibliotecas de aprendizaje automático permite construir una solución equivalente a un costo mucho menor, a cambio de más trabajo de integración.

En el pronóstico de demanda, la práctica tradicional en el comercio minorista son los métodos estadísticos por serie, como el suavizamiento exponencial y, para demanda intermitente, el método de Croston (1972) y su corrección de Syntetos y Boylan (2005). En los últimos años, los modelos globales de aprendizaje automático entrenados con muchas series a la vez, en particular los basados en árboles con gradient boosting, se han vuelto una alternativa competitiva, porque aprovechan patrones compartidos entre productos y sucursales e incorporan con facilidad variables de calendario y precio (Hyndman y Athanasopoulos, 2021).

En calidad de datos, la literatura distingue varias dimensiones, como completitud, exactitud, consistencia y unicidad, que deben medirse explícitamente en lugar de suponerse (Wang y Strong, 1996; Batini y Scannapieco, 2016). DATACORE retoma estas ideas: mide cada regla de calidad contra una verdad conocida y cada pronóstico contra métodos de referencia.

# CAPÍTULO II MARCO TEÓRICO

## MARCO TEÓRICO

## 2.1 Ingeniería de datos y arquitectura por capas

La ingeniería de datos se ocupa de diseñar y operar los sistemas que obtienen, almacenan, transforman y entregan datos para su análisis. Un sistema de datos confiable debe ser correcto aun ante fallas, escalable cuando crece el volumen y mantenible por quienes lo heredan (Kleppmann, 2017). Para lograrlo se organiza por capas con responsabilidades separadas: una capa de fuentes, una de procesamiento, una de persistencia, una de analítica y una de servicios y visualización. Separar las capas permite cambiar una pieza, por ejemplo el motor de visualización, sin rehacer las demás.

Dos propiedades atraviesan todas las capas. La reproducibilidad garantiza que el mismo código con los mismos datos de entrada produce exactamente los mismos resultados, lo que permite verificar cualquier cifra. La trazabilidad permite seguir un dato desde el reporte final hasta su archivo y fila de origen.

## 2.2 Procesos ETL

Un proceso ETL (extracción, transformación y carga) reúne datos de fuentes heterogéneas, los lleva a un formato común y los carga en un repositorio central (Kimball y Ross, 2013). En la extracción se leen archivos con distintas codificaciones, separadores y estructuras. En la transformación se limpian y estandarizan los valores, se aplican reglas de negocio y se resuelven las llaves: los códigos que usan los sistemas de origen se traducen a los identificadores internos del repositorio. En la carga, los registros validados se escriben en la base de datos, de preferencia en una sola transacción para que una falla a la mitad no deje datos incompletos.

Un ETL bien diseñado es idempotente: correrlo dos veces sobre los mismos archivos produce el mismo resultado y no duplica registros. También deja una bitácora de cada ejecución, con las filas leídas, cargadas y rechazadas, y de cada hallazgo de calidad.

## 2.3 Calidad de datos

La calidad de datos se define como la aptitud de los datos para el uso que se les quiere dar (Wang y Strong, 1996). Se evalúa en varias dimensiones, resumidas en la Tabla 2.1.

**Tabla 2.1.** *Dimensiones de la calidad de datos*

| Dimensión | Pregunta que responde | Ejemplo de defecto |
|---|---|---|
| Completitud | ¿Están todos los valores requeridos? | Un cliente sin correo electrónico. |
| Unicidad | ¿Cada entidad aparece una sola vez? | El mismo ticket registrado dos veces. |
| Validez | ¿Los valores cumplen el formato y el dominio esperados? | Una fecha en otro formato o un correo sin arroba. |
| Consistencia | ¿Los datos relacionados concuerdan entre sí? | Un total que no coincide con la suma de sus renglones. |
| Exactitud | ¿El valor refleja la realidad? | Un precio capturado con el punto decimal mal puesto. |
| Integridad referencial | ¿Las referencias apuntan a entidades que existen? | Un renglón de venta con un producto inexistente. |

*Nota.* Elaboración propia con base en Wang y Strong (1996) y Batini y Scannapieco (2016).

Una regla de calidad revisa una de estas dimensiones y, ante un registro defectuoso, toma una acción: lo corrige si el valor correcto se puede recuperar, lo rechaza si no se puede usar, o lo marca para revisión si es utilizable pero sospechoso. Cuando se conoce cuáles registros tienen defecto, el desempeño de una regla se mide con dos métricas tomadas de la recuperación de información. La precisión indica qué fracción de los registros que la regla señaló eran realmente defectuosos; la exhaustividad, o recall, indica qué fracción de los registros defectuosos fue señalada.

$$\text{Precisión} = VP / (VP + FP) \qquad \text{Exhaustividad} = VP / (VP + FN) \qquad (2.1)$$

donde VP son los verdaderos positivos, FP los falsos positivos y FN los falsos negativos. Cuando se necesita un solo número que equilibre ambas métricas se usa la medida F1, su media armónica:

$$F1 = 2 \cdot \text{Precisión} \cdot \text{Exhaustividad} / (\text{Precisión} + \text{Exhaustividad}) \qquad (2.2)$$

## 2.4 Modelo relacional y PostgreSQL

El modelo relacional, propuesto por Codd (1970), organiza los datos en relaciones o tablas, cada una con una llave primaria que identifica sus filas de manera única.

Las llaves foráneas expresan los vínculos entre tablas y permiten al gestor de base de datos garantizar la integridad referencial: no se puede registrar una venta en una sucursal que no existe. La normalización reparte los datos en tablas de forma que cada hecho se guarde en un solo lugar, lo que evita contradicciones.

En los sistemas analíticos se distinguen las tablas de catálogo, que describen entidades como sucursales o productos, de las tablas de hechos, que registran eventos como ventas o movimientos de inventario (Kimball y Ross, 2013). Las vistas son consultas guardadas que se comportan como tablas y permiten calcular indicadores sin duplicar datos. Las vistas materializadas guardan además el resultado, lo que acelera las consultas costosas a cambio de tener que refrescarlas cuando cambian los datos.

PostgreSQL es un gestor de bases de datos relacional de código abierto. Entre sus características útiles para una plataforma de datos están los esquemas, que agrupan tablas por propósito; los roles con permisos diferenciados, que permiten que una aplicación solo lea; el tipo JSONB para guardar estructuras semiestructuradas, y las herramientas pg_dump y pg_restore para respaldar y restaurar.

## 2.5 Análisis exploratorio de datos

El análisis exploratorio de datos (EDA) es la etapa en la que se estudian los datos con estadísticas descriptivas y gráficas antes de modelarlos. Su propósito es entender la forma de los datos, encontrar patrones y anomalías, y orientar las decisiones de modelado. En ventas al menudeo, el EDA suele revisar la tendencia y la estacionalidad, las diferencias entre sucursales y categorías, los patrones de calendario y la concentración de las ventas.

La concentración se analiza con el principio de Pareto, según el cual una fracción pequeña de los productos genera la mayor parte del ingreso. La clasificación ABC aplica esta idea al inventario: los productos A generan alrededor del 80 % del ingreso, los B el siguiente 15 % y los C el resto (Silver et al., 2017). Cada clase merece un nivel distinto de atención.

## 2.6 Pronóstico de demanda

### 2.6.1 Series de tiempo y demanda intermitente

Una serie de tiempo es una sucesión de observaciones ordenadas en el tiempo, como las unidades vendidas de un producto cada semana. Sus componentes típicos son la tendencia, la estacionalidad y el ruido (Hyndman y Athanasopoulos, 2021). Los métodos más sencillos sirven como referencia: el pronóstico ingenuo repite el último valor observado, el estacional repite el valor del mismo periodo del año anterior y la media móvil promedia los últimos periodos.

La demanda de un producto en una sucursal suele ser intermitente: muchos periodos en cero y algunos con venta irregular. Croston (1972) propuso pronosticar por separado el tamaño de la demanda cuando ocurre y el intervalo entre ocurrencias. Syntetos y Boylan (2005) demostraron que ese método tiene un sesgo positivo y propusieron una corrección, conocida como SBA, que multiplica el pronóstico por un factor menor que uno.

### 2.6.2 Gradient boosting con pérdida Poisson

El gradient boosting construye un modelo predictivo como suma de muchos árboles de decisión pequeños, donde cada árbol nuevo se ajusta para corregir los errores que dejan los anteriores (Friedman, 2001). La implementación por histogramas agrupa los valores de cada variable en intervalos, lo que acelera el entrenamiento en conjuntos grandes. La biblioteca scikit-learn ofrece esta implementación como HistGradientBoostingRegressor (Pedregosa et al., 2011).

Cuando la variable a predecir es un conteo no negativo con muchos ceros, como las unidades vendidas por semana, conviene entrenar con pérdida Poisson. Esta pérdida modela el logaritmo de la media esperada, de modo que las predicciones siempre son positivas y el error se pondera de forma adecuada para conteos. Un solo modelo global entrenado con todas las series aprende patrones compartidos, como el efecto del Buen Fin en una categoría, y ayuda a pronosticar las series con poca historia.

### 2.6.3 Validación temporal y fuga de información

Un modelo de pronóstico debe evaluarse con datos que no vio durante el entrenamiento y respetando el orden del tiempo. La validación con origen móvil (rolling origin) entrena el modelo con los datos hasta una fecha de corte, lo evalúa en el bloque siguiente y repite el procedimiento avanzando el corte (Tashman, 2000). Así se obtienen varias mediciones fuera de muestra en distintas épocas del año.

El error más grave en esta evaluación es la fuga de información: que alguna variable del modelo use datos posteriores al momento en que se haría el pronóstico. Un modelo con fuga parece excelente en la evaluación y falla en producción. Para pronosticar a h semanas, toda variable debe calcularse con datos hasta la semana t − h.

### 2.6.4 Métricas de error

El error porcentual absoluto medio (MAPE) no está definido cuando el valor real es cero, lo que lo vuelve inútil para demanda intermitente (Hyndman y Koehler, 2006). En su lugar se usa el error absoluto porcentual ponderado (WAPE), que divide la suma de los errores absolutos entre la suma de los valores reales y, por lo tanto, pondera cada error por el volumen:

$$WAPE = \Sigma\, |y_t - \hat{y}_t| \,/\, \Sigma\, y_t \qquad (2.3)$$

El sesgo indica si el modelo tiende a quedarse corto o a pasarse. Un sesgo negativo significa que el modelo subestima la demanda:

$$Sesgo = \Sigma\, (\hat{y}_t - y_t) \,/\, \Sigma\, y_t \qquad (2.4)$$

### 2.6.5 Demanda censurada

Cuando un producto se agota, la venta registrada es menor que la demanda real: los clientes que no encontraron el producto no quedan registrados. Este fenómeno se conoce como demanda censurada. Un modelo entrenado solo con ventas aprende a predecir ventas, no demanda, y por lo tanto la subestima justo en los productos que más se agotan. Las estrategias para atenuarlo incluyen excluir del

entrenamiento los periodos con faltante o informar al modelo, mediante variables de disponibilidad, cuándo la venta estuvo limitada.

### 2.6.6 Intervalos de pronóstico

Un pronóstico puntual no indica cuánto puede desviarse la demanda real. Para decidir inventarios se necesitan intervalos: un rango en el que se espera que caiga la demanda con cierta probabilidad. Para conteos con más variabilidad de la que admite la distribución de Poisson, se usa la distribución binomial negativa, cuya varianza crece con el cuadrado de la media:

$$Var(Y) = \mu + \mu^2 / k \qquad (2.5)$$

donde μ es la media pronosticada y k es un parámetro de dispersión que se estima a partir de los errores históricos. Con esta distribución se obtienen percentiles, por ejemplo el percentil 90 de la demanda durante el tiempo de reposición.

## 2.7 Gestión de inventarios

La gestión de inventarios busca equilibrar dos costos: el de quedarse sin producto, que se traduce en ventas perdidas y clientes insatisfechos, y el de tener producto de más, que inmoviliza capital, ocupa espacio y arriesga mermas (Silver et al., 2017). En una política (s, S) se revisa la existencia periódicamente y, cuando cae por debajo del punto de reorden s, se pide lo necesario para llegar al nivel máximo S.

El punto de reorden debe cubrir la demanda esperada durante el tiempo de entrega del proveedor más un inventario de seguridad que absorba la variabilidad. La cobertura de inventario expresa cuántos días alcanza la existencia actual al ritmo de venta esperado; una cobertura muy alta indica exceso y una cobertura cero, desabasto.

## 2.8 Detección de anomalías

Una anomalía es una observación que se aleja de lo esperado más de lo que explica la variación normal. Su detección requiere dos piezas: un valor esperado, calculado solo con datos pasados, y una prueba que decida si la diferencia es demasiado grande. Para conteos, como el número de tickets diarios de una sucursal, la prueba

puede basarse en la cola de la distribución de Poisson: si la probabilidad de observar un valor tan bajo es menor que un umbral, por ejemplo 1 %, el día se marca como anómalo.

Para montos, que no son conteos, se usan medidas robustas como la mediana y la desviación absoluta mediana (MAD), que no se distorsionan por valores extremos (Rousseeuw y Croux, 1993). Un puntaje z robusto mide cuántas MAD se aleja una observación de la mediana.

## 2.9 Servicios web y visualización

Una API REST expone datos y operaciones por medio del protocolo HTTP: cada recurso tiene una ruta y se consulta con métodos estándar como GET o POST. Separar la API del resto del sistema permite que distintas aplicaciones consuman los mismos datos sin acceder directamente a la base y que las reglas de seguridad se apliquen en un solo lugar. FastAPI es un marco de Python para construir APIs que valida automáticamente los parámetros de entrada y genera documentación interactiva.

Un dashboard presenta indicadores, tendencias y alertas en forma visual para apoyar decisiones. Su valor depende de que muestre pocas cifras relevantes con claridad, en lugar de muchas gráficas sin jerarquía. Streamlit es un marco de Python que permite construir aplicaciones web de datos con poco código.

## 2.10 Operación de plataformas de datos

Una plataforma de datos en producción necesita mecanismos que aseguren su continuidad. Un respaldo solo es útil si se puede restaurar, por lo que la práctica recomendada es probar la restauración de forma periódica y verificar que los datos restaurados coincidan con los originales. El monitoreo revisa que cada servicio responda y que los procesos programados se hayan ejecutado a tiempo, y las bitácoras registran lo ocurrido para diagnosticar fallas.

La integración continua (CI) ejecuta automáticamente las pruebas del sistema con cada cambio al código, de modo que un error se detecta en minutos y no semanas después (Fowler, 2006). Los contenedores, como los de Docker, empaquetan una

aplicación con todas sus dependencias para que se ejecute igual en la computadora de desarrollo y en el servidor (Merkel, 2014).

## 2.11 Objetivos de Desarrollo Sostenible

La Agenda 2030 de las Naciones Unidas establece 17 Objetivos de Desarrollo Sostenible (Organización de las Naciones Unidas, 2015). Dos de ellos se relacionan con este proyecto. El ODS 9, Industria, innovación e infraestructura, promueve infraestructura resiliente y la modernización tecnológica de las empresas. El ODS 12, Producción y consumo responsables, busca reducir el desperdicio a lo largo de las cadenas de suministro, lo que en el comercio incluye evitar el exceso de inventario que termina en mermas o en productos que no se venden.

# CAPÍTULO III DEFINICIÓN DEL PROYECTO

## DEFINICIÓN DEL PROYECTO

## 3.1 Planteamiento del problema

NexaTech Solutions administra información de cinco áreas distintas sin una plataforma central que la integre, la valide y la analice de forma automática. Esta situación dificulta consultar el histórico, detectar inconsistencias, dar seguimiento al inventario y generar indicadores confiables para la toma de decisiones. Los datos llegan con duplicados, valores faltantes, formatos inconsistentes y errores de captura, y nadie mide cuántos hay ni de qué tipo.

La empresa tampoco cuenta con mecanismos automatizados para supervisar la calidad de los datos, respaldar la información, monitorear los servicios que la administran ni anticipar cambios en la demanda. El resurtido se decide con estimaciones manuales que no consideran la estacionalidad ni la demanda que se pierde por faltantes. Como consecuencia, alrededor de 4.7 % de la demanda queda sin surtir mientras otros productos acumulan existencia para más de seis meses.

La pregunta que guía el proyecto es la siguiente: ¿cómo puede una empresa mediana con datos dispersos y defectuosos convertirlos, de forma automática y verificable, en indicadores, pronósticos y alertas que apoyen sus decisiones de inventario?

## 3.2 Justificación

La información empresarial solo tiene valor cuando se puede almacenar, consultar y analizar de manera confiable. Un proyecto de datos no puede limitarse a la etapa de modelado: también debe resolver cómo se obtienen los registros, cómo se validan, dónde se almacenan, cómo se protegen y cómo se convierten en información útil. DATACORE integra esas etapas en un solo sistema y permite aplicar conocimientos de bases de datos, programación, procesamiento de información, análisis exploratorio, aprendizaje automático, despliegue y administración de servicios.

El uso de datos simulados aporta una ventaja metodológica: se conoce la verdad. El simulador registra cada defecto que inyecta y la demanda real de cada día, incluida la que no se surtió. Gracias a ello, cada componente se puede evaluar con números objetivos en lugar de suponer que funciona.

Desde el punto de vista del negocio, un mejor pronóstico y un sistema de alertas permiten reducir tanto el desabasto como el exceso de inventario. Desde el punto de vista social, el proyecto se alinea con dos Objetivos de Desarrollo Sostenible:

- ODS 9, Industria, innovación e infraestructura: la plataforma usa infraestructura digital, análisis de datos e inteligencia artificial para modernizar la gestión de la información de una empresa.
- ODS 12, Producción y consumo responsables: el pronóstico de demanda y las alertas de exceso y de productos sin rotación apoyan una planeación de compras que reduce el inventario inmovilizado, las mermas y las compras innecesarias.

## 3.3 Alcances y restricciones

El alcance contempla un prototipo funcional de principio a fin. La Tabla 3.1 resume lo que el proyecto incluye y lo que queda fuera.

**Tabla 3.1.** *Alcance del proyecto*

| Incluye | No incluye |
|---|---|
| Generador de datos simulados reproducible, con defectos controlados y su bitácora. | Integración con sistemas reales de punto de venta, CRM o almacén. |
| ETL con reglas de calidad medibles y base de datos PostgreSQL. | Facturación y procesos contables. |
| Análisis exploratorio e indicadores de ventas, inventario e incidencias. | Autenticación de usuarios finales con roles complejos. |
| Modelo de pronóstico de demanda y alertas de inventario. | Aplicación móvil. |
| API, dashboard, respaldos, bitácoras y monitoreo. | Procesamiento en tiempo real; la carga es por lotes, una vez al día. |
| Contenedor, integración continua y manuales técnico y de usuario. | Compra automática: las alertas apoyan la decisión, no la sustituyen. |

*Nota.* Elaboración propia.

El proyecto se desarrolló con las siguientes restricciones:

- Todos los datos son simulados. No se usa información de personas ni de empresas reales, y las marcas de los productos son inventadas.
- El periodo simulado va del 1 de enero de 2023 al 31 de diciembre de 2025.
- El equipo de desarrollo es una computadora con Windows 11 y unos 16 GB de memoria, sin Docker ni compilador de C instalados. Por eso se usó Python

3.15 con paquetes precompilados y la imagen de Docker se construye en el servicio de integración continua.

- Solo se usan herramientas de código abierto o gratuitas.
- El despliegue debe hacerse con recursos sin costo para el equipo, como los créditos estudiantiles de un servicio de nube, y solo con la aprobación del equipo.
- La duración del proyecto es de 16 semanas.

## 3.4 Objetivo General

Desarrollar una plataforma inteligente para la centralización, administración, análisis y monitoreo de la información empresarial de NexaTech Solutions S.A. de C.V., mediante una arquitectura de datos que automatice el procesamiento de la información, asegure su integridad y genere herramientas analíticas y predictivas que apoyen la toma de decisiones sobre ventas e inventarios.

## 3.5 Objetivos específicos

1. Diseñar una arquitectura de software y datos que centralice la información generada por las distintas áreas de la empresa simulada.
2. Diseñar e implementar una base de datos relacional para administrar la información de productos, clientes, sucursales, ventas, proveedores e inventarios.
3. Generar conjuntos de datos simulados con características y comportamientos similares a los observados en operaciones empresariales reales.
4. Desarrollar procesos ETL para la extracción, limpieza, transformación, validación y carga automática de la información.
5. Implementar reglas de calidad que identifiquen valores faltantes, duplicados, registros inconsistentes y posibles anomalías.
6. Desarrollar un módulo de análisis y aprendizaje automático que estime la demanda de productos e identifique riesgos de desabasto o de exceso de inventario.

7. Implementar un dashboard que permita consultar indicadores, tendencias, alertas, resultados analíticos y predicciones.
8. Configurar mecanismos de respaldo, registro de eventos y monitoreo básico de los servicios de la plataforma.
9. Desplegar e integrar los componentes del sistema en un entorno centralizado de ejecución.
10. Documentar la arquitectura, el funcionamiento, las pruebas y los resultados del proyecto para facilitar su uso y mantenimiento.

# CAPÍTULO IV DESARROLLO DEL PROYECTO

## DESARROLLO DEL PROYECTO

## 4.1 Desarrollo de las etapas

### 4.1.1 Metodología y organización del trabajo

El proyecto se organizó en ocho fases de dos semanas, cada una con un entregable concreto que se versionó junto con su documentación. La Figura 4.1 muestra el cronograma.

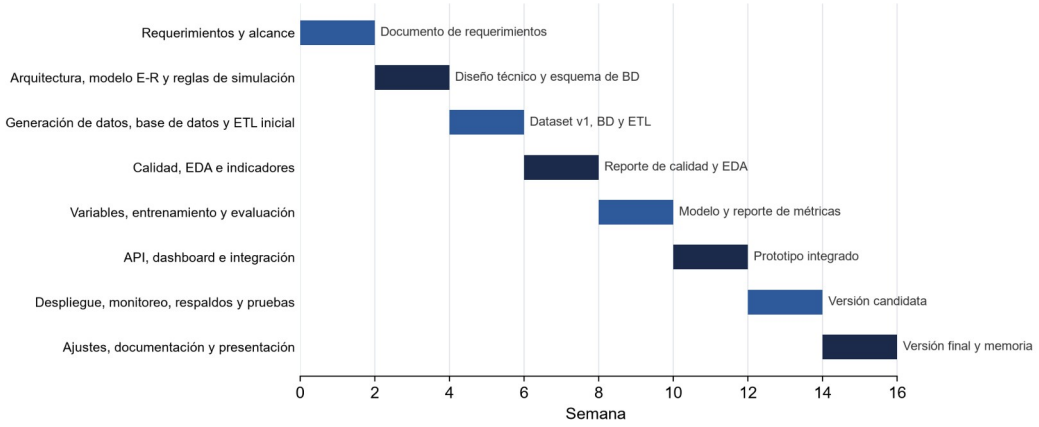



**Figura 4.1.** *Cronograma del proyecto en fases de dos semanas*

*Nota.* Elaboración propia con base en la propuesta del proyecto.

El proyecto se desarrolló en equipo. Al cierre de cada fase, los avances, las decisiones de diseño y los resultados se revisaron en conjunto antes de pasar a la siguiente, de modo que ambos integrantes conocieran el funcionamiento completo de la solución: desde la simulación de datos y el proceso ETL hasta los modelos de pronóstico, la API y el dashboard.

El trabajo se apoyó en cuatro prácticas:

- Control de versiones. El código y la documentación viven en un repositorio privado de GitHub. Cada cambio se registra con un mensaje descriptivo, lo que deja un historial auditable del avance.
- Documentación continua. Cada decisión de diseño se registró en la carpeta docs del repositorio en el momento de tomarla, en lugar de dejar toda la documentación para el final. Los seis documentos técnicos y los dos manuales son la base de este reporte.

- Pruebas automatizadas. Cada módulo nuevo se entregó con sus pruebas, y la integración continua las ejecuta con cada cambio.
- Asistentes de programación con inteligencia artificial. Se usaron agentes de código (Claude y Codex) como herramienta para acelerar tareas acotadas, como el análisis exploratorio, las vistas de indicadores, la detección de anomalías y varias páginas del dashboard. Un archivo AGENTS.md en el repositorio fija sus reglas: convenciones de código, estructura del proyecto y restricciones del equipo. Todo cambio propuesto por un agente pasó por revisión humana y por las pruebas automatizadas antes de integrarse. Durante esa revisión se detectaron y corrigieron errores reales, como la elección de la prueba estadística del detector de anomalías y la pérdida del historial de pronósticos al recargar los datos, descritas más adelante.

### 4.1.2 Primera etapa: requerimientos y alcance

En la primera etapa se definieron el problema, el alcance, los supuestos y los riesgos, y se redactaron los requerimientos. Se establecieron 13 requerimientos funcionales, de los cuales 9 son de prioridad alta (Tabla 4.1), y 8 requerimientos no funcionales (Tabla 4.2).

**Tabla 4.1.** *Requerimientos funcionales*

| ID | Requerimiento | Prioridad |
|---|---|---|
| RF-01 | Generar un dataset simulado reproducible de unos 3 años, con defectos controlados y su bitácora. | Alta |
| RF-02 | Ingerir automáticamente los archivos de todas las áreas (CSV, Excel, distintas codificaciones). | Alta |
| RF-03 | Limpiar, estandarizar y validar los datos con reglas de calidad configurables. | Alta |
| RF-04 | Registrar cada hallazgo de calidad y cada ejecución del ETL. | Alta |
| RF-05 | Cargar los datos validados en una base relacional con integridad referencial. | Alta |
| RF-06 | Calcular indicadores de ventas, ticket promedio, rotación, cobertura y desabasto. | Alta |
| RF-07 | Pronosticar la demanda semanal por producto y sucursal. | Alta |
| RF-08 | Generar alertas de desabasto, exceso de inventario y productos sin rotación. | Alta |
| RF-09 | Exponer datos, indicadores, predicciones y alertas mediante una API. | Media |
| RF-10 | Ofrecer un dashboard interactivo con filtros por fecha, sucursal y categoría. | Alta |
| RF-11 | Respaldar la base de datos automáticamente y poder restaurarla. | Media |
| RF-12 | Monitorear la disponibilidad de los servicios y registrar eventos. | Media |
| RF-13 | Detectar anomalías en ventas o inventario (complementario). | Baja |

*Nota.* Elaboración propia.

**Tabla 4.2.** *Requerimientos no funcionales*

| ID | Requerimiento |
|---|---|
| RNF-01 | Reproducibilidad: el mismo código con la misma semilla produce exactamente los mismos datos. |
| RNF-02 | Rendimiento: el ETL completo termina en menos de 10 minutos. |
| RNF-03 | Trazabilidad: todo registro rechazado o corregido puede rastrearse hasta su archivo y fila de origen. |
| RNF-04 | Idempotencia: volver a correr el ETL sobre los mismos archivos no duplica datos. |
| RNF-05 | Calidad medible: se reportan la precisión y la exhaustividad de las reglas de calidad. |
| RNF-06 | Seguridad básica: credenciales fuera del código y un usuario de base de datos de solo lectura. |
| RNF-07 | Mantenibilidad: código en Python con pruebas automatizadas y documentación al día. |
| RNF-08 | Portabilidad: ejecución en Windows (desarrollo) y en Linux (servidor). |

*Nota.* Elaboración propia.

También se identificaron los riesgos principales y su mitigación (Tabla 4.3). El más relevante era que los datos simulados resultaran poco realistas y volvieran trivial el modelo; para evitarlo, las reglas de simulación incluyen estacionalidad, tendencias, demanda censurada, productos nuevos y descontinuados.

**Tabla 4.3.** *Riesgos del proyecto y su mitigación*

| Riesgo | Probabilidad | Impacto | Mitigación |
|---|---|---|---|
| Datos simulados poco realistas que vuelvan trivial el modelo | Media | Alto | Reglas de simulación documentadas con estacionalidad, tendencias, censura y ciclo de vida. |
| Bibliotecas sin paquetes precompilados para Python 3.15 | Media | Medio | Verificación temprana de dependencias; Python 3.13 como alternativa. |
| No contar con un servidor a tiempo | Media | Medio | Contenedor independiente del proveedor y plan de despliegue documentado. |
| Carga de trabajo desigual entre integrantes | Baja | Medio | Frentes divididos con revisión cruzada. |
| Alcance demasiado amplio para 16 semanas | Media | Alto | Priorizar los requerimientos de prioridad alta; RF-13 como opcional. |

*Nota.* Elaboración propia.

### 4.1.3 Segunda etapa: arquitectura y diseño de la base de datos

La arquitectura se organizó en cinco capas (Figura 4.2). El generador produce los archivos de cada área; el ETL los valida y los carga en PostgreSQL; el módulo analítico escribe pronósticos y alertas; la API los sirve, y el dashboard los presenta al usuario. Un proceso programado ejecuta cada noche la cadena completa.

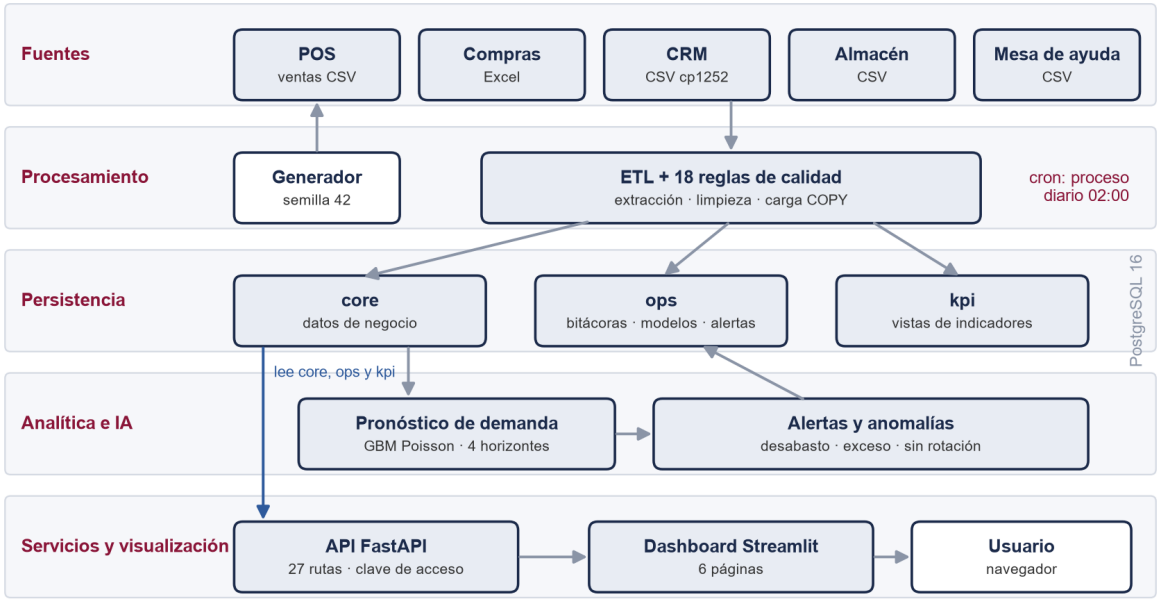



**Figura 4.2.** *Arquitectura por capas de DATACORE*

*Nota.* Elaboración propia.

La Tabla 4.4 muestra la tecnología elegida para cada capa y la alternativa que se había considerado en la propuesta. Se eligió una combinación en la que todo es Python y de código abierto, de modo que un solo lenguaje cubre del generador al dashboard.

**Tabla 4.4.** *Tecnologías por capa*

| Capa | Función | Tecnología elegida | Alternativa considerada |
|---|---|---|---|
| Fuentes | Archivos simulados por área | Python con numpy, pandas y Faker | — |
| Procesamiento | Extracción, limpieza, validación y carga | Python con pandas | — |
| Persistencia | Almacenamiento relacional | PostgreSQL 16 | MySQL |
| Analítica e IA | EDA, entrenamiento, evaluación y predicción | scikit-learn, numpy y pandas | — |
| Servicios | Comunicación entre la base, el modelo y la interfaz | FastAPI | Flask |
| Visualización | Indicadores, alertas y tendencias | Streamlit | Power BI |
| Infraestructura | Ejecución, monitoreo y respaldos | Docker, GitHub Actions y cron | VPS dedicado |

*Nota.* Elaboración propia.

Las decisiones de diseño más importantes se registraron con su motivo (Tabla 4.5). Destaca la segunda: el generador guarda una verdad de referencia limpia que nunca lee el ETL ni el modelo, y que solo se usa para evaluarlos.

**Tabla 4.5.** *Decisiones de diseño*

| Decisión | Motivo |
|---|---|
| Los archivos crudos usan códigos de negocio (código de sucursal, SKU), no llaves internas. | Así exporta un sistema real; obliga al ETL a resolver las llaves. |
| El generador guarda una verdad de referencia limpia. | Permite medir objetivamente el ETL y las reglas de calidad. |
| Las ventas se recortan según la existencia disponible. | La demanda no surtida no aparece en las ventas, como en la realidad. |
| Los datos no se versionan; se regeneran con la semilla. | Pesan unos 80 MB y se reproducen en unos 15 segundos. |
| Las tablas de salida del modelo no tienen llave foránea hacia los datos de negocio. | El ETL recarga los datos de negocio por completo y no debe borrar el historial de pronósticos. |

*Nota.* Elaboración propia.

La base de datos se dividió en tres esquemas. El esquema core guarda los datos de negocio ya validados en diez tablas; el esquema ops guarda la operación de la plataforma, y el esquema kpi contiene las vistas de indicadores. La Figura 4.3 presenta el modelo entidad-relación del esquema core y la Tabla 4.6 resume los tres esquemas. El diccionario de datos completo se incluye en el Anexo 1.

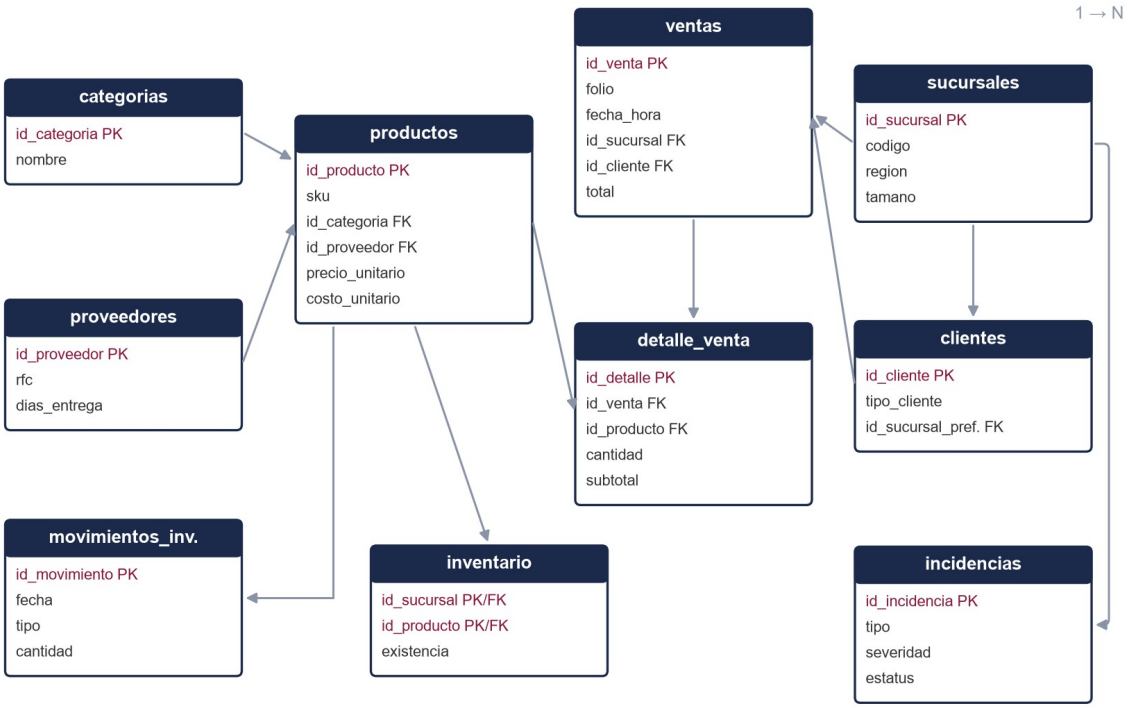



**Figura 4.3.** *Modelo entidad-relación del esquema core*

*Nota.* Elaboración propia. Las flechas van de la tabla referenciada (1) a la que la referencia (N).

**Tabla 4.6.** *Esquemas de la base de datos*

| Esquema | Contenido | Tablas o vistas principales |
|---|---|---|
| core | Datos de negocio validados | sucursales, categorias, proveedores, productos, clientes, ventas, detalle_venta, inventario, movimientos_inventario, incidencias |
| ops | Operación de la plataforma | etl_ejecuciones, calidad_hallazgos, modelos, predicciones_demanda, alertas_inventario, anomalias, respaldos, monitoreo, corridas |
| kpi | Indicadores calculados | ventas_diarias_sucursal, ventas_mensuales_categoria, ventas_semanales_producto, abc_productos, cobertura_inventario, desabasto_mensual, resumen_ejecutivo, entre otras |

*Nota.* Elaboración propia.

Se adoptaron convenciones uniformes: las llaves primarias se llaman id_ seguido del nombre de la entidad; los nombres van en español, sin acentos y en minúsculas separadas por guion bajo; los montos se guardan en pesos mexicanos con dos decimales, y las cantidades de inventario son enteros con signo, positivos cuando el producto entra y negativos cuando sale. Para la seguridad se crearon dos roles: uno dueño de los esquemas, que usan el ETL, el modelo y los respaldos, y otro de solo lectura, que usa la API para sus consultas.

### 4.1.4 Tercera etapa: simulación de datos

El generador de datos produce tres años de operación en unos 15 segundos y siempre da exactamente los mismos datos con la semilla 42, lo que cumple el requerimiento de reproducibilidad. La generación sigue un orden que imita cómo surgen los datos en la realidad:

1. Catálogos: sucursales, categorías, proveedores, productos y clientes.
2. Demanda: tickets y renglones según la afluencia, el calendario y la mezcla de productos.
3. Inventario: simulación día a día; la demanda que no alcanza la existencia se pierde.
4. Ventas finales: los renglones se recortan a lo que realmente se surtió, en orden de llegada.
5. Incidencias: se derivan de eventos reales de la simulación, como faltantes, retrasos y fallas.
6. Exportación: se guarda la verdad de referencia y luego se generan los archivos crudos con defectos.

La Tabla 4.7 muestra el volumen resultante.

**Tabla 4.7.** *Volumen de datos generados con la semilla 42*

| Tabla | Filas |
|---|---|
| Sucursales | 12 |
| Categorías | 10 |
| Proveedores | 35 |
| Productos | 400 |
| Clientes | 8,000 (más 160 casi duplicados en el archivo crudo) |
| Ventas (tickets) | 181,054 |
| Detalle de venta | 640,673 |
| Movimientos de inventario | 647,613 |
| Inventario (pares sucursal-producto) | 4,800 |
| Incidencias | 6,720 |
| Órdenes de compra (oculta) | 124,072 |

*Nota.* Elaboración propia con base en docs/04_simulacion.md.

La demanda sigue reglas explícitas para que los datos tengan la estructura de una operación real y no sean simples valores aleatorios (Tabla 4.8).

**Tabla 4.8.** *Reglas de generación de la demanda*

| Patrón | Regla |
|---|---|
| Afluencia por sucursal | Tickets por día con distribución de Poisson: 19 en sucursal grande, 13 en mediana y 8.5 en pequeña. |
| Maduración | Una sucursal nueva arranca al 45 % de su afluencia y llega al 100 % en unos 6 meses. |
| Día de la semana | Factores de 0.85 el lunes a 1.35 el sábado. |
| Quincena | Los días 1, 15, 16 y fin de mes multiplican la afluencia por 1.15. |
| Fechas especiales | Buen Fin ×1.9 con descuentos; 18 al 24 de diciembre hasta ×1.6; Reyes ×1.5; 10 de mayo ×1.5; 25 de diciembre y 1 de enero ×0.3. |
| Estacionalidad por categoría | Perfiles mensuales: Bebidas en verano, Electrónica a fin de año, Papelería en agosto, Juguetes en diciembre, enero y abril. |
| Popularidad | Distribución lognormal: pocos productos concentran la mayor parte de las ventas. |
| Tendencia | Crecimiento anual por producto con media de 6 % y desviación de 12 %; tráfico +3 % anual. |
| Afinidad regional | Multiplicador de 0.7 a 1.3 por región y categoría. |
| Ciclo de vida | Unos 8 % de los productos se lanzan dentro del periodo y unos 5 % se descontinúan. |
| Clientes | Cada cliente compra 90 % en su sucursal preferida; los mayoristas llevan más renglones y cuatro veces la cantidad. |
| Precios | Lista anual con inflación de alrededor de 5 %, actualizada cada enero. |
| Fallas de sistema | Unas 0.8 fallas por sucursal al mes; ese día la afluencia baja en proporción a las horas sin servicio. |

*Nota.* Elaboración propia con base en docs/04_simulacion.md.

El inventario se simula día a día con una política (s, S) por par sucursal-producto. El punto de reorden cubre la demanda estimada durante el tiempo de entrega más un inventario de seguridad, y el nivel máximo agrega entre 10 y 35 días de cobertura. Para que la simulación produzca los problemas reales que la plataforma debe atacar, la demanda estimada de cada par tiene un error fijo de entre −45 % y +45 %, se recalibra cada 90 días con las ventas observadas y no con la demanda, y los proveedores se retrasan con una probabilidad de 3 a 35 %. También hay mermas en productos perecederos, ajustes por conteo físico al cierre de cada trimestre y productos descontinuados que dejan inventario muerto. Así surgen de forma natural tanto el desabasto como el exceso.

Al exportar los archivos, el generador inyecta defectos controlados y registra cada uno en una bitácora con la tabla, la llave, el tipo, el valor original y el valor sucio. La Figura 4.4 muestra los 18 tipos y su frecuencia. A estos defectos se suman los

problemas estructurales de formato: el archivo de clientes en Windows-1252 con punto y coma, los catálogos en Excel y las ventas partidas por año.

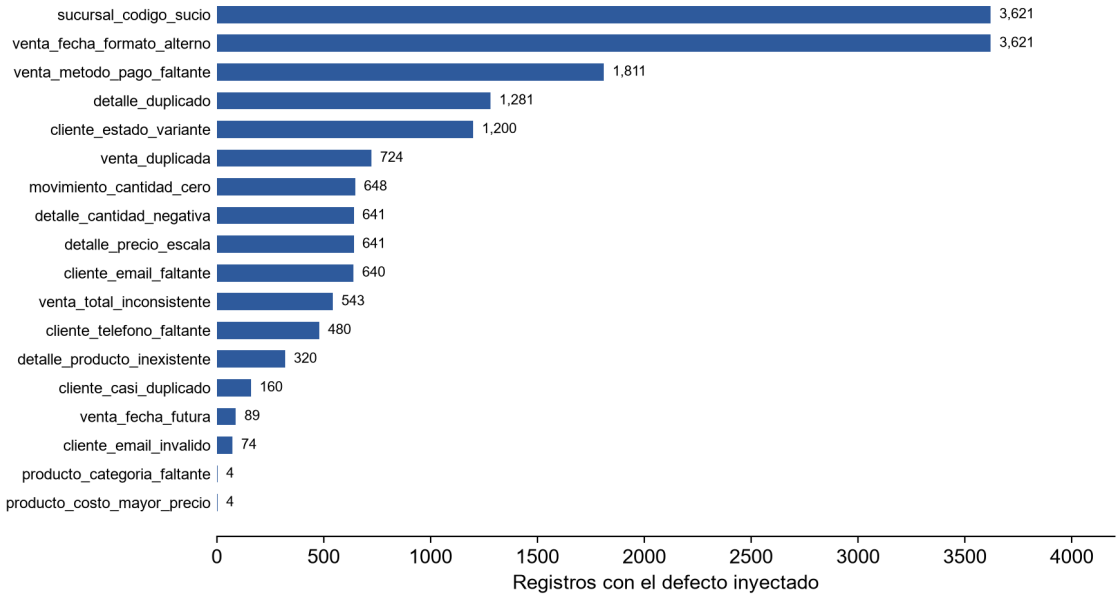



**Figura 4.4.** *Defectos inyectados en los archivos crudos por tipo*

*Nota.* Elaboración propia. Total: 16,502 defectos.

### 4.1.5 Cuarta etapa: ETL y reglas de calidad

El ETL sigue el flujo de la Figura 4.5. Extrae los archivos de las cinco áreas, estandariza los valores, aplica las reglas de calidad, traduce los códigos de negocio a llaves internas, carga los datos en una sola transacción mediante el comando COPY de PostgreSQL y, opcionalmente, evalúa el resultado contra la verdad de referencia.

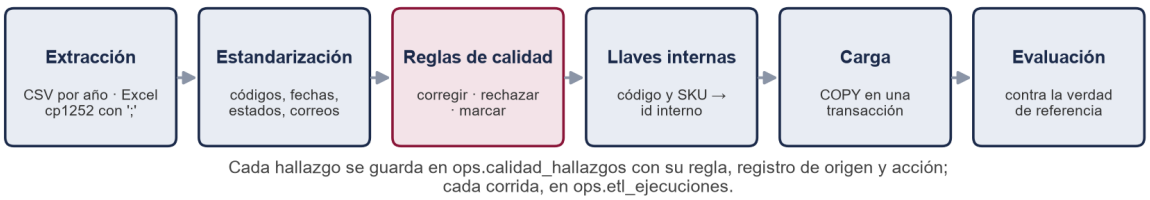



**Figura 4.5.** *Flujo del proceso ETL*

*Nota.* Elaboración propia.

Cada regla registra sus hallazgos en ops.calidad_hallazgos con la tabla, la llave del registro, el nombre de la regla, la acción tomada y los valores antes y después. Las reglas que atacan un defecto del generador llevan el mismo nombre que el tipo de defecto, lo que permite medir automáticamente su precisión y exhaustividad. La Tabla 4.9 resume las reglas principales y su acción.

**Tabla 4.9.** *Reglas de calidad principales del ETL*

| Regla | Tabla | Acción |
|---|---|---|
| venta_duplicada, detalle_duplicado | ventas, detalle_venta | Corregir: se conserva un solo registro. |
| venta_fecha_formato_alterno | ventas | Corregir: se convierte al formato ISO. |
| venta_fecha_futura | ventas | Corregir: se toma el año del archivo anual del que proviene el ticket. |
| venta_metodo_pago_faltante | ventas | Marcar para revisión. |
| venta_total_inconsistente | ventas | Corregir: el total se recalcula con la suma de los renglones. |
| sucursal_codigo_sucio | ventas | Corregir: mayúsculas, sin espacios ni guiones. |
| detalle_cantidad_negativa | detalle_venta | Corregir: se toma el valor absoluto. |
| detalle_precio_escala | detalle_venta | Corregir: se reconstruye a partir del subtotal o se reescala por 10. |
| detalle_producto_inexistente | detalle_venta | Rechazar. |
| cliente_casi_duplicado | clientes | Corregir: se fusiona con el registro original (mismo nombre normalizado y fecha de registro). |
| cliente_email_invalido | clientes | Corregir si es recuperable; si no, marcar. |
| cliente_email_faltante, cliente_telefono_faltante | clientes | Marcar. |
| cliente_estado_variante | clientes | Corregir: se normaliza al nombre oficial del estado. |
| producto_categoria_faltante | productos | Corregir: se infiere por el prefijo del SKU. |
| producto_costo_mayor_precio | productos | Corregir: se imputa el costo con el margen mediano de su categoría. |
| movimiento_cantidad_cero | movimientos_inventario | Corregir con la diferencia de existencias del par; si no se puede, rechazar. |

*Nota.* Elaboración propia con base en src/datacore/etl/transformacion.py.

El ETL no se limita a descartar registros defectuosos: también recupera información. Los clientes casi duplicados se identifican por su nombre normalizado, sin acentos

ni diferencias de mayúsculas, y su fecha de registro; se fusionan con el registro original y sus compras se reasignan a él. Para detectar un precio con el punto decimal mal puesto, se compara contra la mediana del mismo producto en ese año; si el subtotal capturado es coherente, el precio se reconstruye a partir de él, porque multiplicar por 10 no recupera los centavos ya redondeados. Las fechas en formato alterno se convierten y las fechas con un año fuera del archivo anual se corrigen con el año de ese archivo. Cuando se rechaza un renglón de venta, el total del ticket se ajusta para que siga cuadrando con su detalle.

Además de las reglas ligadas a defectos inyectados, el ETL aplica validaciones estructurales que no dependen del generador: llaves inválidas, ventas sin detalle, detalles sin venta, movimientos de inventario que no cuadran con la existencia resultante y estados de cliente desconocidos. Estas reglas protegen la base de datos ante defectos que el simulador no contempla.

La carga completa tarda unos 50 segundos, muy por debajo del límite de 10 minutos del requerimiento RNF-02. Como el ETL recarga el esquema core dentro de una transacción, volver a ejecutarlo sobre los mismos archivos produce exactamente el mismo resultado, lo que cumple el requerimiento de idempotencia. Al terminar, refresca las vistas materializadas del esquema kpi.

### 4.1.6 Quinta etapa: análisis exploratorio e indicadores

Una vez cargados los datos se realizó un análisis exploratorio con el fin de entender su comportamiento y orientar el diseño del modelo. El análisis usa únicamente las diez tablas del esquema core y nunca las tablas ocultas de la verdad de referencia, de modo que reproduce lo que vería un analista en una empresa real. Se analizaron 179,302 ventas completadas por un ingreso total de 285.7 millones de pesos; las 1,752 ventas canceladas se excluyeron de todos los indicadores.

La Figura 4.6 muestra la evolución del ingreso mensual. Se observa una tendencia creciente año con año y un pico pronunciado en noviembre y diciembre. El mes de mayor ingreso fue noviembre de 2025, con 11.5 millones de pesos, y el de menor ingreso, febrero de 2023, con 5.6 millones.

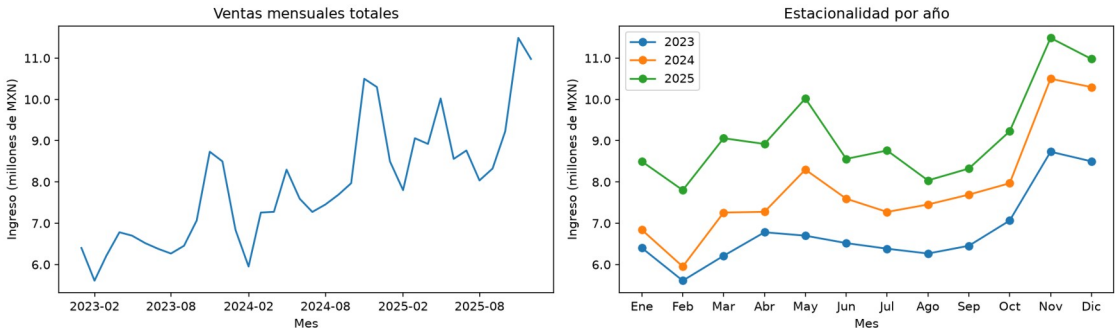



**Figura 4.6.** *Ingreso mensual de NexaTech Solutions, 2023 a 2025*

*Nota.* Elaboración propia con datos simulados.

La estacionalidad difiere por categoría (Figura 4.7). El índice estacional más alto corresponde a Juguetes en diciembre, con 1.98 veces su media mensual; Papelería sube en agosto por el regreso a clases y Electrónica a fin de año. Esta diferencia justificó incluir en el modelo un índice estacional por categoría en lugar de uno general.

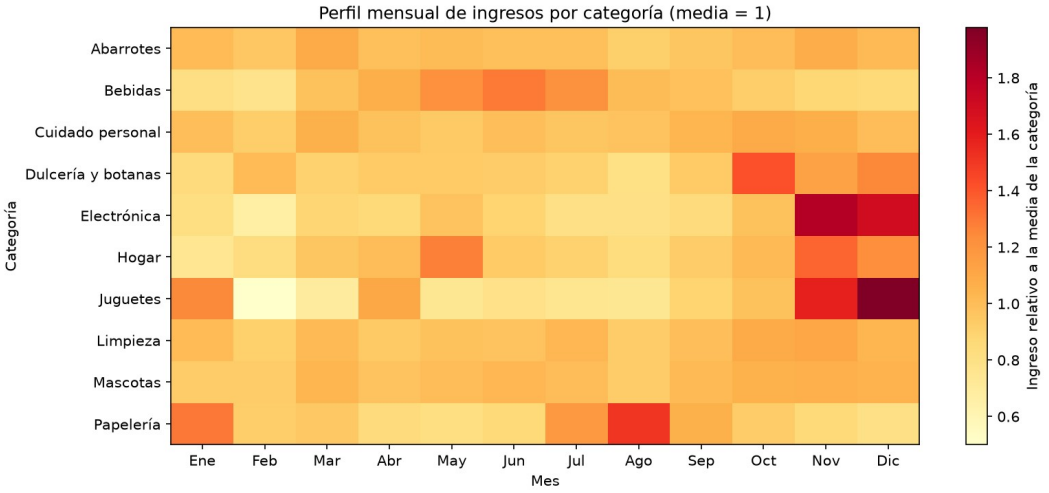



**Figura 4.7.** *Índice estacional por categoría y mes*

*Nota.* Elaboración propia con datos simulados.

Por sucursal (Figura 4.8), CDMX01 encabeza el ingreso con 39.0 millones de pesos, 13.7 % del total, con un ticket promedio de 1,755 pesos. Las sucursales abiertas en 2024 tienen menos ingreso acumulado porque operaron menos tiempo, lo que obliga a comparar por día de operación y a que el modelo considere la antigüedad de cada sucursal.

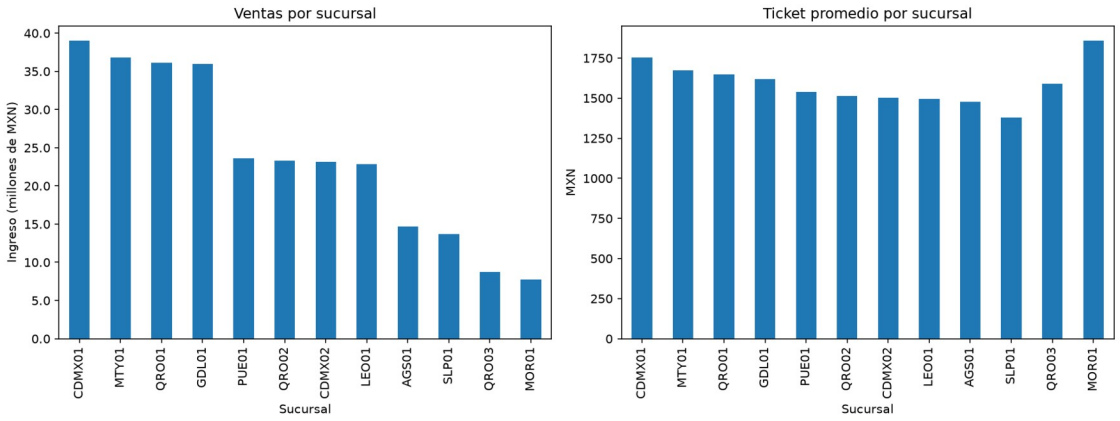



**Figura 4.8.** *Ingreso y ticket promedio por sucursal*

*Nota.* Elaboración propia con datos simulados.

La concentración de las ventas confirma el principio de Pareto (Figura 4.9): 110 de los 400 productos, es decir 27.5 %, generan 80.2 % del ingreso. Estos productos forman la clase A y merecen el pronóstico más preciso, porque un error en ellos cuesta más.

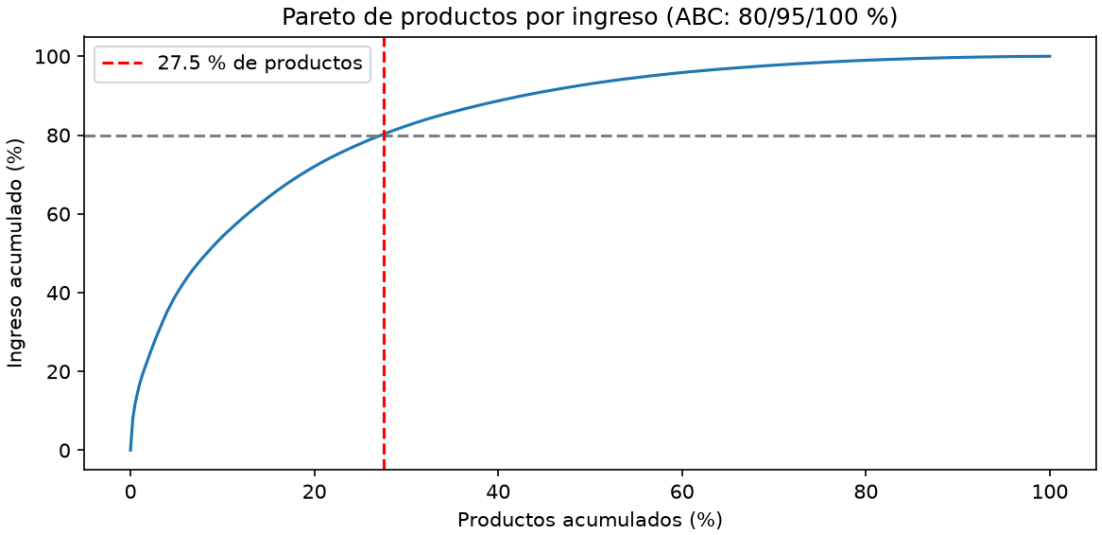



**Figura 4.9.** *Curva de Pareto del ingreso por producto*

*Nota.* Elaboración propia con datos simulados.

Los patrones de calendario (Figura 4.10) muestran que el sábado tiene el mayor ingreso diario medio, 361 mil pesos, y que la hora pico es la una de la tarde. En la mezcla de ventas (Figura 4.11), la transferencia es el método de pago con más ingreso (39.7 %), seguida del efectivo (27.7 %); los mayoristas generan 49.4 % del ingreso, aunque son solo el 10 % de los clientes registrados.

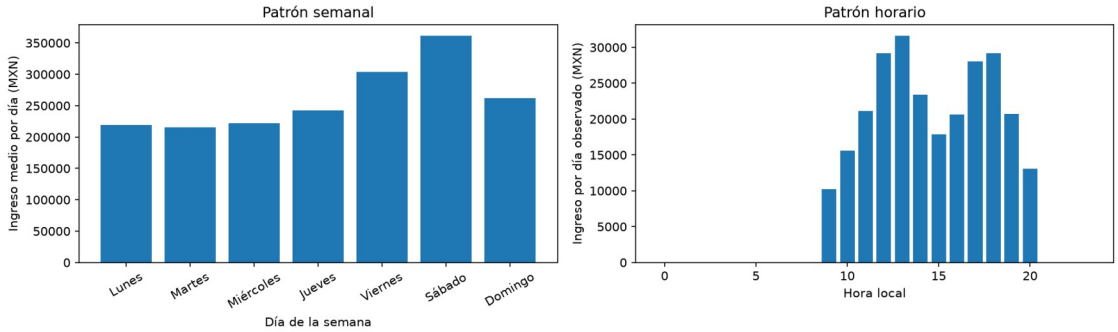



**Figura 4.10.** *Ingreso por día de la semana y por hora*

*Nota.* Elaboración propia con datos simulados.

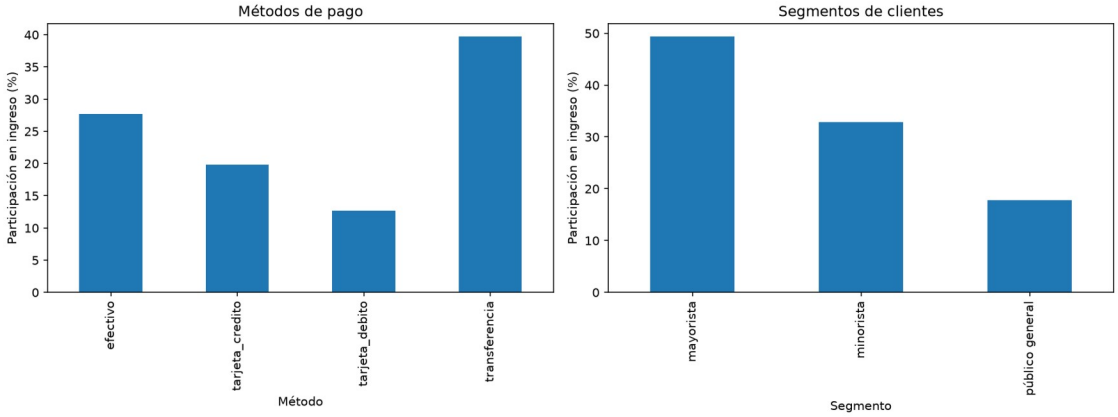



**Figura 4.11.** *Participación en el ingreso por método de pago y segmento de cliente*

*Nota.* Elaboración propia con datos simulados.

El análisis del inventario (Figura 4.12) revela el problema que motiva el proyecto. Al 31 de diciembre de 2025, 1,098 de los 4,800 pares sucursal-producto tienen cobertura mayor a 180 días, incluidos 506 con existencia y sin ventas en 90 días, mientras que 120 pares están sin existencia. La mediana de la cobertura es de 57 días.

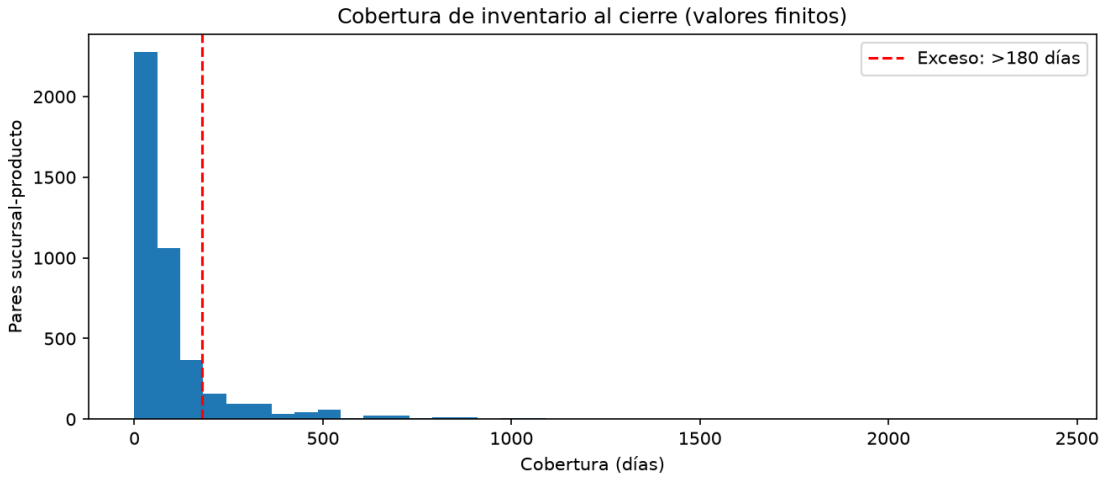



**Figura 4.12.** *Distribución de la cobertura de inventario al cierre de 2025*
*Nota.* Elaboración propia con datos simulados.

Por último, las incidencias (Figura 4.13) están dominadas por los retrasos de proveedor, con 3,180 de 6,720 reportes, y su pico ocurrió en mayo de 2024 con 108.

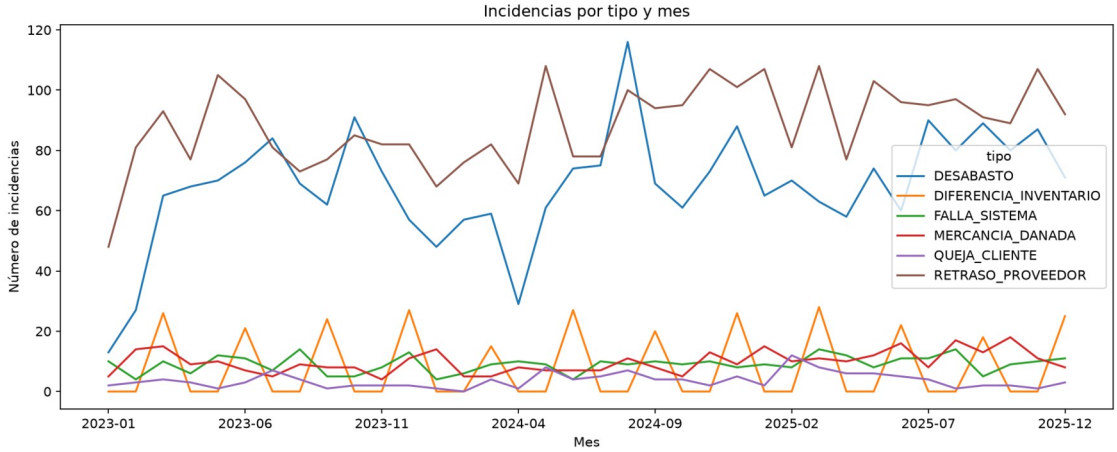



**Figura 4.13.** *Incidencias mensuales por tipo*
*Nota.* Elaboración propia con datos simulados.

El análisis dejó cinco implicaciones para el modelo de demanda. Primera: hay que incorporar mes, día de la semana, quincenas y eventos comerciales, y validar con cortes temporales. Segunda: la demanda es muy intermitente, pues 88.7 % de los días-par de productos activos no tienen venta, por lo que conviene pronosticar por semana y compararse con métodos para series intermitentes. Tercera: hay 44 productos lanzados dentro del periodo y 20 inactivos al cierre, y los días previos al alta no deben interpretarse como demanda cero. Cuarta: las dos sucursales nuevas

requieren modelar su maduración. Quinta: las ventas están censuradas por la disponibilidad, de modo que una existencia en cero no debe confundirse con falta de demanda.

Con base en este análisis se construyó el esquema kpi, cuyas vistas calculan los indicadores que consumen la API y el dashboard (Tabla 4.10). Las vistas filtran solo las ventas completadas, y las dos más costosas se materializan con índices únicos para que las consultas respondan rápido. El ETL las refresca al terminar cada carga.

**Tabla 4.10.** *Vistas de indicadores del esquema kpi*

| Vista | Qué mide |
|---|---|
| ventas_diarias_sucursal | Tickets, unidades, ingreso y ticket promedio diario por sucursal. |
| ventas_mensuales_categoria | Ingreso y unidades por categoría y su participación dentro del mes. |
| ventas_semanales_producto (materializada) | Demanda semanal por sucursal y producto; la semana inicia en lunes. |
| abc_productos | Clasificación de Pareto histórica por ingreso. |
| cobertura_inventario | Existencia actual frente a la demanda de los últimos 90 días y estado del par. |
| desabasto_mensual (materializada) | Proporción de días-par sin existencia en cada mes. |
| incidencias_mensuales | Reportes por tipo y severidad, pendientes y duración media de resolución. |
| calidad_ultima_carga | Hallazgos de calidad por regla y acción de la última carga. |
| resumen_ejecutivo | Una fila con ventas históricas y recientes, crecimiento, clientes, inventario e incidencias. |

*Nota.* Elaboración propia con base en db/vistas.sql.

### 4.1.7 Sexta etapa: modelo de pronóstico de demanda

El módulo de inteligencia artificial pronostica cuántas unidades de cada producto se venderán en cada sucursal durante las próximas cuatro semanas. La Tabla 4.11 resume las decisiones de diseño del modelo.

**Tabla 4.11.** *Decisiones de diseño del modelo de demanda*

| Decisión | Elección | Motivo |
|---|---|---|
| Unidad de pronóstico | Unidades por semana (lunes a domingo), sucursal y producto | Por día, 89 % de los casos no tiene venta; por semana baja a 57 % y coincide con el ciclo de compras. |
| Horizontes | 1 a 4 semanas, un modelo por horizonte | Cubre el tiempo de entrega más la revisión semanal sin acumular errores de pronósticos encadenados. |
| Algoritmo | HistGradientBoostingRegressor con pérdida Poisson, un modelo global para los 4,800 pares | Maneja conteos con muchos ceros, comparte patrones entre pares y entrena en alrededor de un minuto por horizonte. |

| Decisión | Elección | Motivo |
|---|---|---|
| Validación | Origen móvil: 4 bloques de 13 semanas de 2025 | Mide el error en todas las épocas del año, incluidos Buen Fin y Navidad. |
| Métricas | WAPE y sesgo | El MAPE no está definido con ceros; el WAPE pondera por volumen. |

*Nota.* Elaboración propia con base en docs/05_modelo_demanda.md.

El modelo usa 37 variables agrupadas en cinco familias (Tabla 4.12). Para el horizonte h, todo rezago o ventana se calcula con datos hasta la semana t − h, y una prueba automatizada verifica que ninguna variable use información del futuro.

**Tabla 4.12.** *Variables del modelo de demanda*

| Familia | Variables |
|---|---|
| Historia del par | Rezagos de 1 a 4 semanas; misma semana del año anterior, simple y suavizada; medias móviles de 4, 8, 13 y 26 semanas; desviación estándar; máximo; fracción de semanas en cero; suavizamiento exponencial; media histórica; pronóstico SBA. |
| Disponibilidad | Fracción de días sin existencia en la semana de origen y en las últimas 4 semanas. |
| Contexto | Precio cobrado; media del producto en todas las sucursales; índice estacional de la categoría. |
| Calendario | Semana del año, mes, Buen Fin, Navidad, fin de año, Reyes, Día de las Madres, Día del Niño, regreso a clases y número de quincenas en la semana. |
| Atributos | Sucursal, categoría, región, tamaño de sucursal, días de entrega del proveedor y semanas de vida del par. |

*Nota.* Elaboración propia con base en docs/05_modelo_demanda.md.

Las variables de disponibilidad merecen una mención especial. Permiten al modelo distinguir entre una semana sin venta porque nadie pidió el producto y una semana sin venta porque no había producto. Sin ellas, el modelo aprendería que los productos que se agotan tienen poca demanda, justo lo contrario de lo que ocurre.

Para saber si el modelo aporta valor se comparó contra cuatro métodos de referencia: el pronóstico ingenuo (la semana anterior), el estacional (la misma semana del año anterior), la media móvil de 8 semanas y el método SBA para demanda intermitente. Un modelo de aprendizaje automático solo se justifica si supera a estos métodos simples.

Además del pronóstico puntual, el modelo produce un intervalo del 80 % con una distribución binomial negativa, cuya dispersión se estima en la validación. Para las alertas se calcula la demanda durante el tiempo de reposición, que es la suma de los pronósticos semanales prorrateados a los días de entrega del proveedor más

una semana de revisión, junto con su percentil 90. Con estos valores se aplican las reglas de la Tabla 4.13.

**Tabla 4.13.** *Reglas de las alertas de inventario*

| Alerta | Regla | Acción sugerida |
|---|---|---|
| RIESGO_DESABASTO | Existencia actual menor que el percentil 90 de la demanda en el tiempo de reposición (regla original; en la mejora 2 se sustituyó por la demanda esperada con corrección de censura, sección 4.1.13). | Adelantar el pedido al proveedor. |
| EXCESO_INVENTARIO | Cobertura mayor a 180 días al ritmo pronosticado. | Frenar compras, promover o traspasar a otra sucursal. |
| SIN_ROTACION | Existencia positiva, cero ventas en 13 semanas y pronóstico menor que 0.25 unidades por semana. | Evaluar liquidar o descontinuar. |

*Nota.* Elaboración propia con base en docs/05_modelo_demanda.md.

El comando de publicación entrena los cuatro modelos con toda la historia, pronostica las siguientes cuatro semanas, genera las alertas y registra la versión del modelo con sus métricas en ops.modelos. Los pronósticos se guardan en ops.predicciones_demanda y las alertas en ops.alertas_inventario. El proceso completo tarda unos ocho minutos.

Como componente complementario (RF-13) se implementó un detector de anomalías en las ventas diarias de cada sucursal. El valor esperado de cada día se calcula solo con datos pasados: la mediana de los últimos ocho días iguales de la semana, ajustada por la tendencia de los 28 días previos. Sobre los tickets, que son conteos, se aplica una prueba de Poisson y se marca el día si la probabilidad de observar un valor tan bajo es menor que 1 %. Sobre el ingreso se calcula también un puntaje z robusto con MAD, que se publica como información adicional. Las fechas especiales del calendario se excluyen, porque con solo tres años de historia no hay referencias suficientes para ellas.

La primera versión del detector, propuesta por un agente de código, usaba MAD tanto para el ingreso como para los tickets. En la revisión se cambió la prueba de los tickets por la de Poisson, más adecuada para conteos, lo que casi duplicó su desempeño, como se muestra en la sección de resultados.

### 4.1.8 Séptima etapa: API y dashboard

La API se construyó con FastAPI y expone 27 rutas agrupadas en seis temas (Tabla 4.14). Su documentación interactiva se genera automáticamente en la ruta /docs. Todas las rutas, salvo /salud, exigen una clave de acceso enviada en el encabezado X-API-Key cuando esta se configura. Las consultas usan el rol de base de datos de solo lectura y SQL parametrizado, lo que evita la inyección de código; solo la acción de marcar una alerta como atendida usa el rol con permiso de escritura. Los parámetros de entrada, como fechas, sucursales y límites de filas, se validan antes de llegar a la base, y cada petición queda registrada en la bitácora con su método, ruta, código de respuesta y latencia.

**Tabla 4.14.** *Rutas de la API agrupadas por tema*

| Tema | Rutas principales |
|---|---|
| Operación | /salud, /etl/ejecuciones, /calidad/ultima-carga, /calidad/hallazgos, /anomalias, /operacion/monitoreo, /operacion/respaldos, /operacion/corridas, /incidencias/mensuales, /incidencias/abiertas |
| Catálogos | /catalogos/sucursales, /catalogos/categorias, /catalogos/productos |
| Indicadores | /kpi/resumen |
| Ventas | /ventas/diarias, /ventas/sucursales, /ventas/categorias, /ventas/semanales, /productos/abc |
| Inventario | /inventario/cobertura, /inventario/estados, /inventario/desabasto, /alertas, /alertas/{id}/atender |
| Pronóstico | /modelo, /pronosticos, /pronosticos/resumen |

*Nota.* Elaboración propia con base en src/datacore/api/app.py.

El dashboard se construyó con Streamlit y consume únicamente la API, nunca la base de datos directamente. Así, cualquier regla de seguridad o de cálculo se aplica en un solo lugar. Se diseñó una identidad visual propia con los colores del proyecto, componentes reutilizables y una paleta validada para que las gráficas se lean con claridad. El dashboard tiene seis páginas, descritas en la Tabla 4.15.

**Tabla 4.15.** *Páginas del dashboard*

| Página | Contenido | Usuario principal |
|---|---|---|
| Inicio | Indicadores de los últimos 12 meses, salud del inventario, ingreso mensual por año, categorías, sucursales y estado de la plataforma. | Dirección |
| Ventas | Filtros de periodo y sucursal; serie diaria con media móvil; patrón semanal; ventas por sucursal; mezcla por categoría; clasificación ABC. | Dirección y comercial |
| Inventario | Pares por estado; distribución por sucursal; mapa de calor del desabasto mensual; tabla de cobertura con filtros. | Almacén |

| Página | Contenido | Usuario principal |
|---|---|---|
| Alertas | Alertas pendientes por tipo, sucursal y categoría; acción para marcarlas como atendidas; composición por sucursal. | Compras y almacén |
| Pronóstico | Búsqueda de producto; historia de 52 semanas y pronóstico de 4 con intervalo del 80 %; demanda por sucursal; precisión del modelo. | Compras |
| Calidad y operación | Hallazgos de la última carga; ejecuciones del ETL; anomalías; incidencias; estado de respaldos, monitoreo y corridas. | Sistemas |

*Nota.* Elaboración propia.

La página de Inicio (Figura 4.14) funciona como resumen ejecutivo. Abre con un encabezado tipo folio que muestra el corte de datos y una franja con el ingreso, los tickets, el ticket promedio y los clientes activos de los últimos 12 meses, con su variación frente al año anterior. Debajo, una barra reparte los 4,800 pares sucursal-producto por estado de inventario, seguida de las alertas pendientes de cada tipo. Después aparece el mapa de la red de sucursales, en el que el tamaño de cada punto es su ingreso y un anillo rojo marca las tres con más alertas. La página cierra con una tarjeta por sucursal y el ingreso mensual del año en curso contra los dos anteriores.

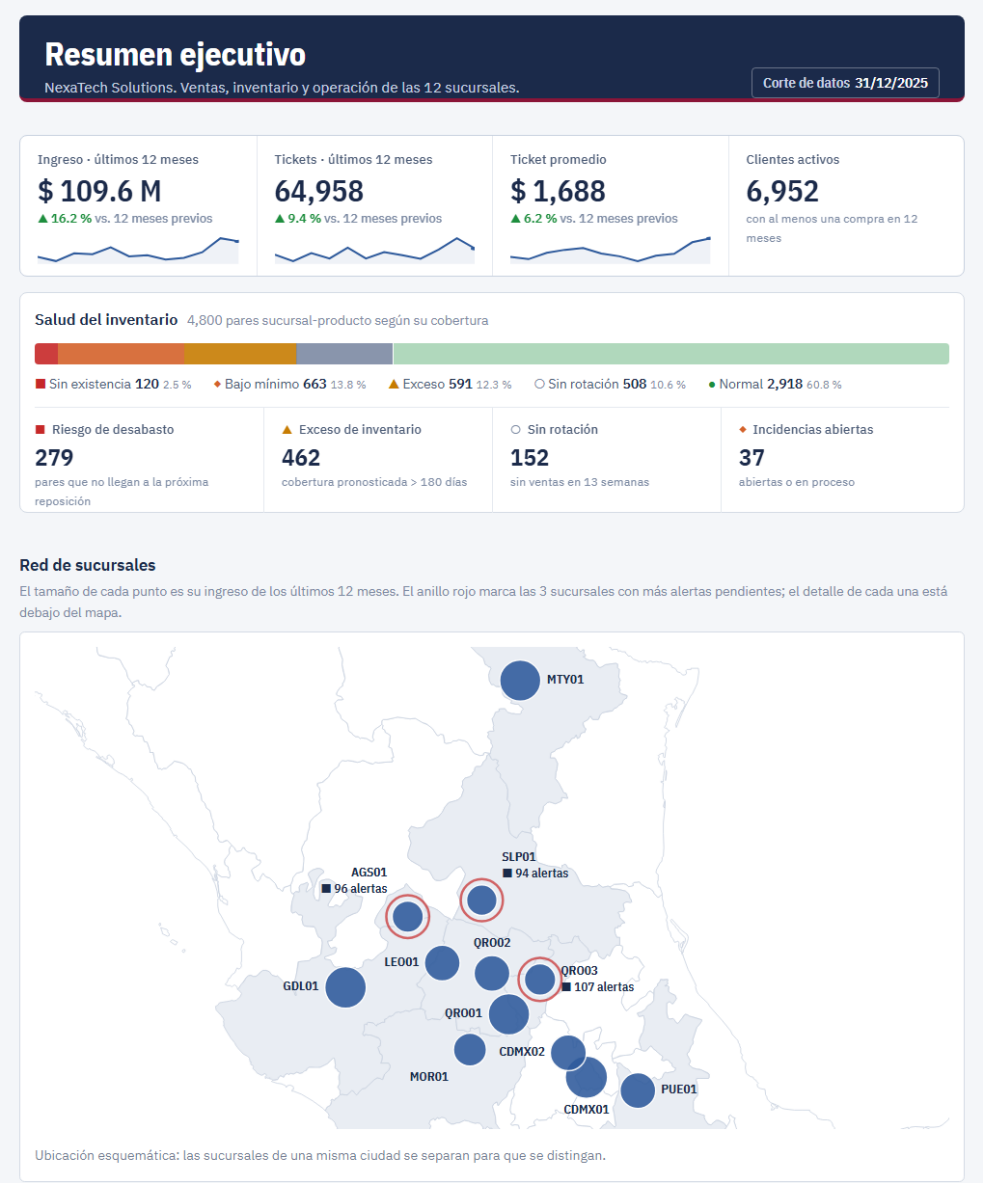



**Figura 4.14.** *Página de Inicio del dashboard*

*Nota.* Elaboración propia. Captura con datos simulados al 31 de diciembre de 2025; se omite la barra lateral de navegación.

La página de Ventas (Figura 4.15) permite elegir el periodo y la sucursal. Muestra el ingreso diario con su media móvil de siete días, el patrón por día de la semana, el ranking de sucursales y la mezcla por categoría.




**Figura 4.15.** *Página de Ventas del dashboard*

*Nota.* Elaboración propia. Captura con el filtro de los últimos 30 días; se muestra la parte superior de la página.

La página de Inventario (Figura 4.16) clasifica los 4,800 pares en cinco estados: sin existencia, bajo mínimo, exceso, sin rotación y normal. Incluye un mapa de calor del desabasto mensual por sucursal en el que se distingue la apertura de las sucursales QRO03 y MOR01.

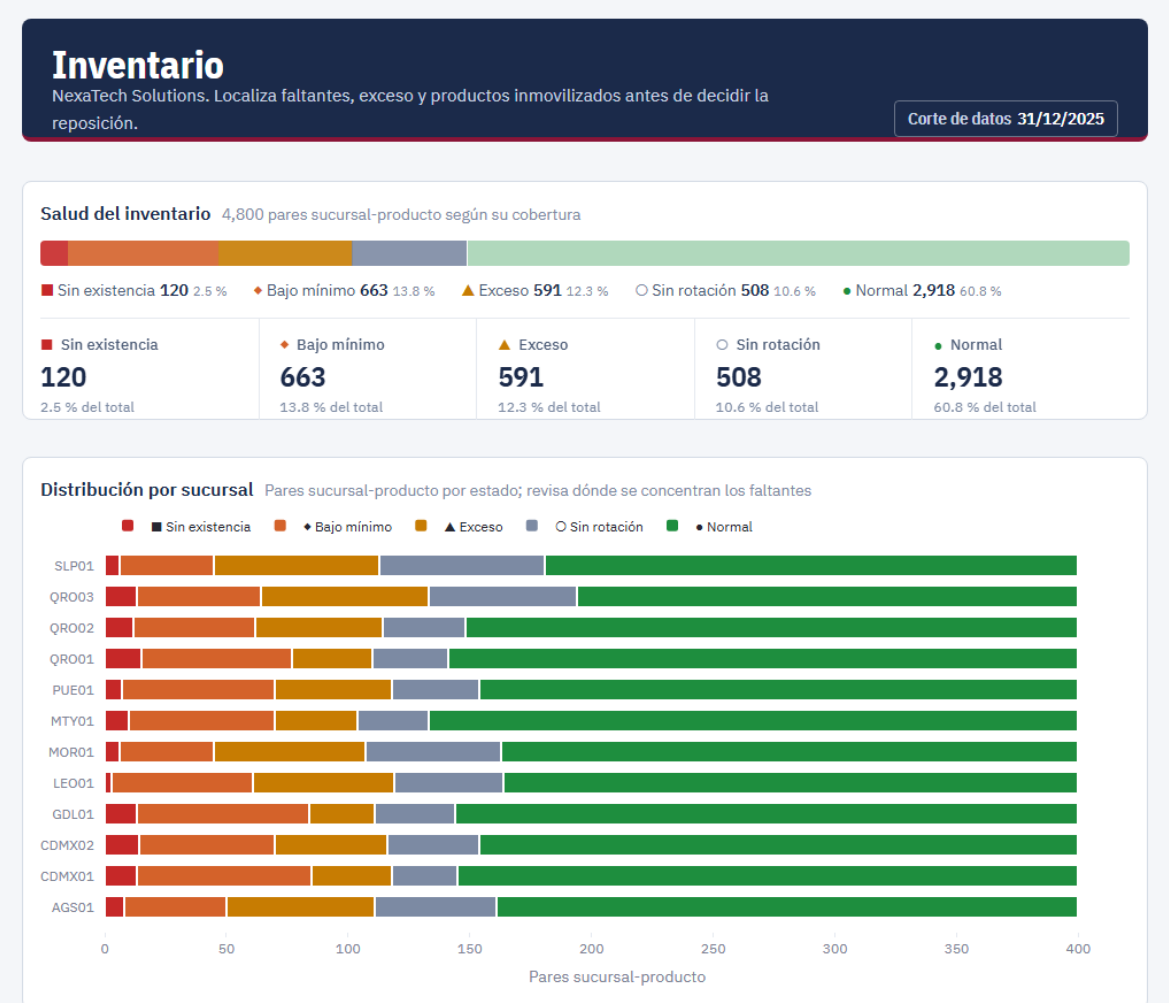



**Figura 4.16.** *Página de Inventario del dashboard*

*Nota.* Elaboración propia. Se muestra la parte superior de la página.

La página de Alertas (Figura 4.17) es la herramienta operativa principal. Lista las alertas pendientes; las de desabasto traen la cantidad sugerida y la fecha límite de pedido, y todas su motivo principal. Al seleccionar filas aparece la explicación completa de cada alerta. La página permite filtrar, marcar como atendidas las alertas revisadas y ver su composición por sucursal para repartir el trabajo.

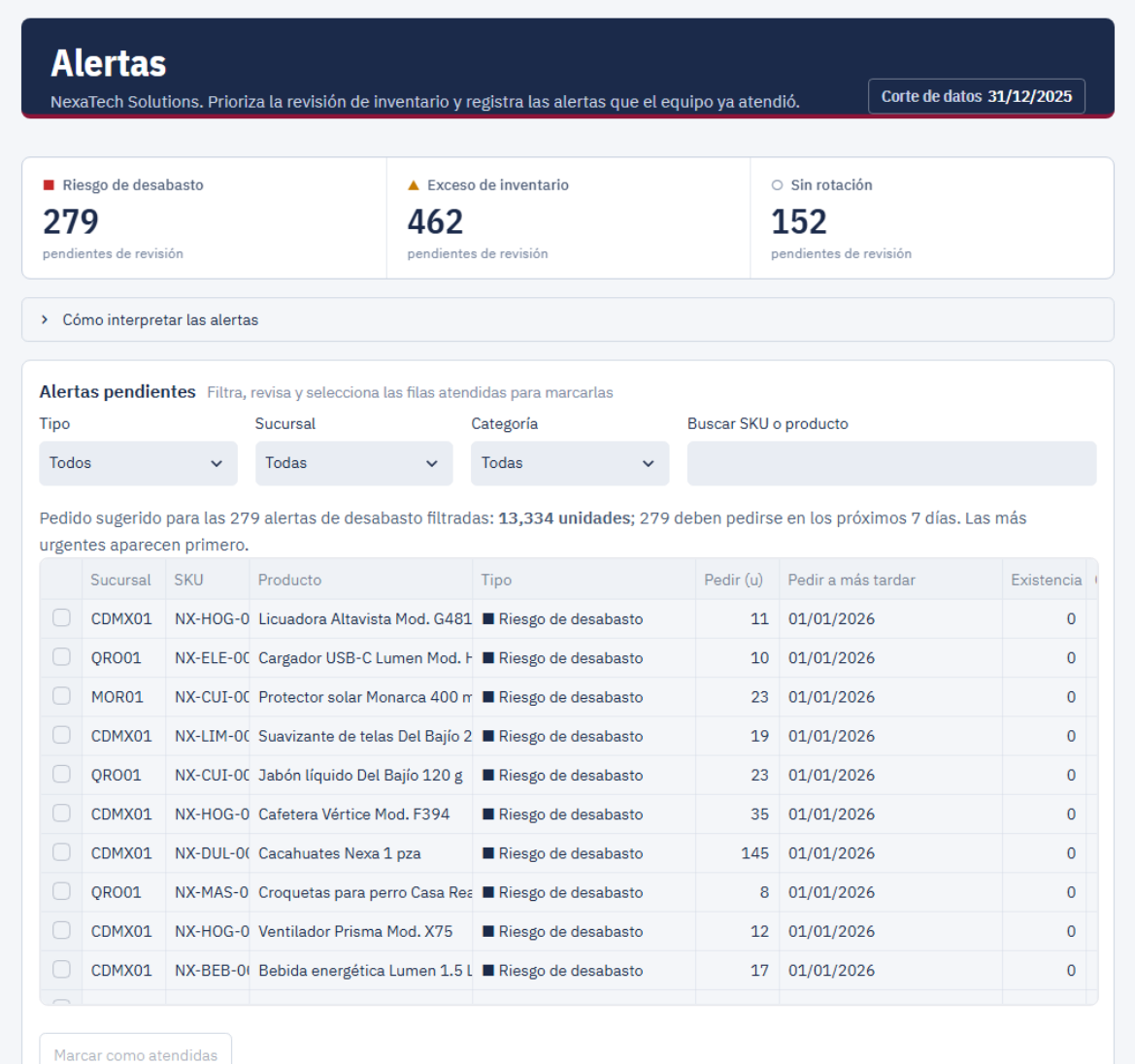



**Figura 4.17.** *Página de Alertas del dashboard*

*Nota.* Elaboración propia. Se muestra la parte superior de la página.

La página de Pronóstico (Figura 4.18) permite buscar un producto y ver su venta de las últimas 52 semanas junto con el pronóstico de las próximas cuatro y su banda de incertidumbre. También desglosa la demanda esperada por sucursal y compara la precisión del modelo con la media móvil.

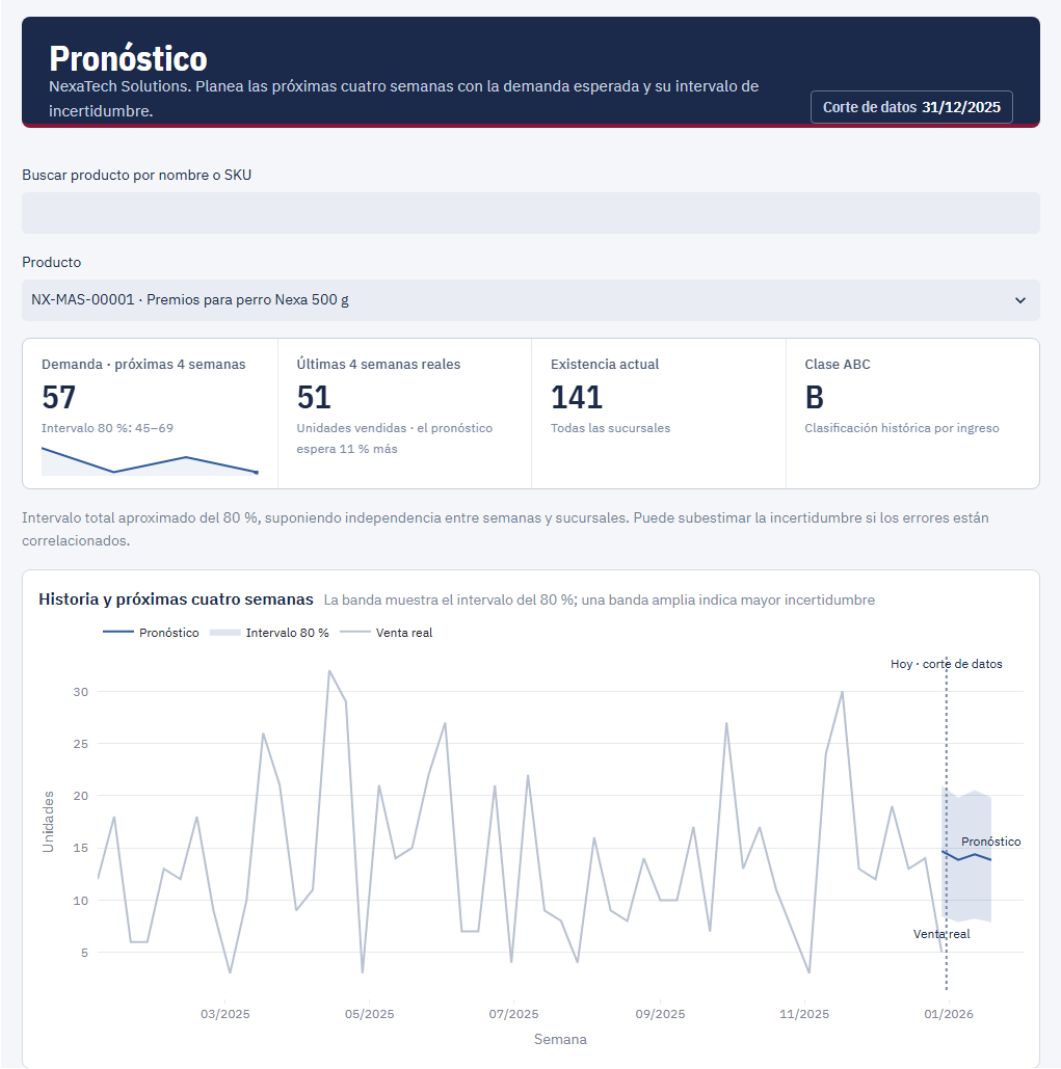



**Figura 4.18.** *Página de Pronóstico del dashboard*

*Nota.* Elaboración propia. Se muestra la parte superior de la página.

La página de Calidad y operación (Figura 4.19) muestra cuántos registros corrigió, marcó o rechazó cada regla en la última carga y el resultado de cada ejecución del ETL, entre otras pestañas sobre anomalías, incidencias y el estado de la plataforma.




**Figura 4.19.** *Página de Calidad y operación del dashboard*

*Nota.* Elaboración propia. Se muestra la parte superior de la página.

### 4.1.9 Octava etapa: operación, respaldos y monitoreo

Para que la plataforma funcione sin intervención manual se implementaron los mecanismos de la Tabla 4.16.

**Tabla 4.16.** *Mecanismos de operación de la plataforma*

| Mecanismo | Funcionamiento | Evidencia |
|---|---|---|
| Respaldo verificado | pg_dump de los esquemas core, ops y kpi; restauración en una base temporal y comparación de filas tabla por tabla; se conservan los 7 más recientes. | ops.respaldos y logs/respaldo.log |
| Monitoreo | Revisa la base de datos, la API, el dashboard, el espacio en disco y la antigüedad del último ETL, respaldo y modelo; devuelve error si algo falla. | ops.monitoreo y logs/monitoreo.log |
| Proceso diario | Respaldo, ETL, publicación del modelo, anomalías y monitoreo, en ese orden. | ops.corridas y logs/diario.log |

| Mecanismo | Funcionamiento | Evidencia |
|---|---|---|
| Bitácoras | Un archivo por componente con rotación a 5 MB y 5 copias. | Carpeta logs |
| Integración continua | Pruebas completas con PostgreSQL 16 y construcción de la imagen Docker en cada cambio. | GitHub Actions |
| Contenedor | Un solo Dockerfile; la variable SERVICIO elige entre API, dashboard, proceso diario o monitoreo. | Dockerfile |

*Nota.* Elaboración propia con base en docs/06_operacion_y_despliegue.md.

La Figura 4.20 muestra el orden del proceso diario. El respaldo va antes del ETL porque el ETL recarga el esquema core por completo; de esta forma siempre existe una copia previa a cualquier cambio. Si un paso falla, la corrida se detiene y el error queda registrado.




**Figura 4.20.** *Pasos del proceso diario automatizado*
*Nota.* Elaboración propia.

El monitoreo compara la antigüedad de cada proceso con umbrales configurables: alerta si el último ETL o el último respaldo tienen más de 26 horas, si el modelo activo tiene más de 8 días o si quedan menos de 5 GB libres en disco. Para la programación automática se preparó un script que registra la tarea diaria a las 2:00 y el monitoreo cada 30 minutos en Windows, y un archivo cron equivalente para Linux.

Durante la revisión de esta etapa se encontró un error importante. Las tablas de pronósticos, alertas y anomalías tenían llaves foráneas hacia el esquema core, y como el ETL recarga core con TRUNCATE en cascada, cada carga borraba también el historial de pronósticos y las alertas ya atendidas. La corrección consistió en quitar esas llaves foráneas, agregar una migración para las bases existentes y una prueba automatizada que impide que vuelvan a aparecer.

La integración continua se configuró en GitHub Actions y se ejecuta con cada cambio a la rama principal. Levanta PostgreSQL 16 como servicio, genera un conjunto de datos reducido de 120 productos y 1,500 clientes, ejecuta el ETL con su evaluación y la publicación del modelo, levanta la API y corre todas las pruebas. Después prueba el respaldo con su restauración y el monitoreo, construye la imagen Docker, la arranca contra la misma base y consulta sus rutas. Al final publica el reporte de calidad y las bitácoras como evidencia.

El despliegue se preparó durante la etapa de mejoras: el equipo eligió Azure for Students y se corrigió la imagen para que funcione en un servidor nuevo (sección 4.1.15). Como alternativa se documentó un servidor privado virtual con Docker o con el archivo cron de Linux.

### 4.1.10 Pruebas del sistema

Cada módulo cuenta con pruebas automatizadas escritas con pytest. Las pruebas que requieren la base de datos, la API o el dashboard se ejecutan completas en la integración continua; al cierre de la etapa de mejoras el proyecto reunía 116 pruebas aprobadas. La Tabla 4.17 resume lo que verifica cada archivo de pruebas.

**Tabla 4.17.** *Pruebas automatizadas por módulo*

| Archivo | Qué verifica |
|---|---|
| test_generator.py | Reproducibilidad con la semilla, llaves foráneas, que el total de cada venta sea la suma de sus renglones, que el inventario cuadre con sus movimientos y que las salidas de inventario igualen a las ventas. |
| test_etl.py | Que se detecte cada tipo de defecto, que las tablas no tengan duplicados ni llaves inválidas, que los totales cuadren y la fidelidad contra la referencia. |
| test_kpi.py | Que las vistas coincidan con cálculos independientes: ingreso diario, unidades semanales, clases ABC, tasa de desabasto y resumen ejecutivo. |
| test_ml.py | Ausencia de fuga de información, que el modelo supere al pronóstico ingenuo, la validación con origen móvil, la evaluación de alertas y el método SBA. |
| test_anomalias.py | Que el detector solo use datos pasados y respete el calendario y las aperturas, y la evaluación sin duplicar métricas. |
| test_api.py | Que todas las rutas respondan, la validación de filtros, la clave de acceso, la atención de alertas y el intervalo agregado del pronóstico. |
| test_dashboard.py | Que las páginas funcionen con la API real y muestren un aviso claro cuando la API no está disponible. |

| Archivo | Qué verifica |
|---|---|
| test_operacion.py | Respaldo y recuperación, retención de respaldos, monitoreo y que ops no dependa de core por llaves foráneas. |
| test_eda.py | Que el reporte exploratorio se genere completo. |
| test_impacto.py, test_alertas_segmentadas.py | Motor de simulación, calibración del cuantil y evaluación anidada de las alertas sin usar el bloque evaluado. |
| test_recomendacion.py, test_explicaciones.py | Cantidad sugerida, fecha límite y factores del porqué de cada alerta. |
| test_datos_iniciales.py | Que un servidor nuevo genere los datos solo cuando faltan. |

*Nota.* Elaboración propia con base en la carpeta tests del repositorio.

Una prueba particularmente importante es la que verifica la ausencia de fuga de información en el modelo: altera los datos posteriores a una fecha de corte y comprueba que las variables calculadas hasta esa fecha no cambian. Si alguna variable dependiera del futuro, la prueba fallaría.

### 4.1.11 Novena etapa: de plataforma que funciona a plataforma que demuestra su valor

Con la plataforma completa y probada, el equipo se planteó una pregunta distinta: ¿cuánto vale DATACORE para el negocio? Que el modelo pronostique mejor que una media móvil no dice cuánto dinero ahorra ni qué debe hacer un comprador con una alerta. Para responderlo se trabajó en cinco mejoras, cada una con su propio commit, sus pruebas y su documentación: medir el impacto en dinero, mejorar la alerta de desabasto, convertir cada alerta en una recomendación de pedido, explicar el porqué de cada alerta y preparar el despliegue. En paralelo se rediseñó el dashboard.

En todas las mejoras se aplicó la misma regla de trabajo: ninguna decisión se toma mirando el periodo con el que se evalúa. Cuando una idea no se sostuvo fuera de muestra, se documentó y no se publicó.

### 4.1.12 Mejora 1: impacto en dinero

El módulo ml/impacto.py simula día a día el año 2025 sobre la misma demanda real de cada par sucursal-producto, es decir, las ventas más la demanda que no se surtió por falta de existencia, que el simulador registra. Sobre esa demanda se comparan dos políticas de resurtido:

- Política actual: se reproducen las órdenes de compra que NexaTech emitió en 2025, con sus fechas de llegada reales. Antes de usar el simulador se verificó que reproduce lo ocurrido: con las órdenes reales da 46,227 unidades perdidas contra 47,976 registradas; la diferencia de 3.6 % se debe a que se omiten mermas y ajustes de conteo en ambas políticas.
- Política DATACORE: cada domingo, con el pronóstico fuera de muestra del backtest, se calcula un cuantil de la demanda durante el tiempo de entrega, el retraso esperado del proveedor y la semana de revisión, y se pide lo necesario para cubrirlo, contando la existencia y lo ya pedido.

La primera versión de la política DATACORE perdió más ventas que la política actual. El análisis de los pares con más pérdidas mostró la causa: el modelo aprende de ventas, y un producto que pasó semanas agotado parece venderse poco, cuando en realidad no había qué vender. Es la demanda censurada descrita en el marco teórico. Se corrigió dividiendo el pronóstico entre (1 − fracción de días agotado en las últimas cuatro semanas), con un tope de ×2, información que DATACORE sí tiene al momento de decidir.

El cuantil de reposición no se eligió mirando 2025. Se extendió el backtest a siete bloques, desde abril de 2024, y se calibró en abril a diciembre de 2024 como el menor cuantil con el que DATACORE iguala el nivel de servicio de la política actual. El resultado fue el cuantil 99, que después se aplicó fijo a 2025. Que se necesite un cuantil nominal tan alto indica que la incertidumbre por par está subestimada, algo coherente con el WAPE por par cercano a 77 %.

Los supuestos de dinero son configurables y quedan listados en el reporte reports/ml/impacto.md: precio y costo de la lista vigente sin descuentos, margen igual a precio menos costo y un costo anual de mantener inventario de 25 % del costo, que agrupa capital, almacén, seguro y obsolescencia. Los resultados se presentan en la sección 4.2.8.

### 4.1.13 Mejora 2: alertas de desabasto más precisas

La alerta original disparaba cuando la existencia era menor que el percentil 90 de la demanda durante la entrega más una semana. Tenía 15.6 % de precisión: cerca de

cinco alertas falsas por cada acierto. El módulo ml/alertas_segmentadas.py comparó cinco reglas con una evaluación anidada: cada trimestre de 2025 se evalúa con reglas elegidas solo con los trimestres anteriores, de modo que el periodo evaluado nunca participa en la elección.

- Regla p90, la publicada hasta entonces.
- Regla de la media: la existencia contra la demanda esperada, sin colchón.
- Umbral único afinado: el mejor cuantil, elegido con los trimestres previos.
- Umbral por segmento: un cuantil distinto por clase ABC y por tiempo de entrega (corto, medio y largo).
- Media con corrección de censura, la misma corrección de la mejora 1.

La regla final se decidió con los bloques de 2024 y se confirmó en los cuatro de 2025. Ganó la media con corrección de censura. El umbral por segmento, en cambio, no se sostuvo: con nueve segmentos y pocos faltantes en cada uno, los umbrales se ajustaron al ruido de los trimestres de entrenamiento, por lo que no se publicó. La regla nueva redujo las alertas de desabasto pendientes de 591 a 279. Los resultados detallados están en la sección 4.2.9.

### 4.1.14 Mejora 3: de alertar a recomendar

Una alerta indica un riesgo, pero no dice qué hacer. Para cada alerta de desabasto se calculan ahora tres datos, que se guardan junto con la alerta en ops.alertas_inventario:

- Nivel objetivo: el cuantil 99 de la demanda durante la entrega más una semana (binomial negativa, corregida por censura), es decir, la demanda esperada más un stock de seguridad. Es el mismo cuantil calibrado en la mejora 1.
- Cantidad sugerida: el nivel objetivo menos la existencia, redondeado hacia arriba, con un mínimo de una unidad.
- Fecha límite: la fecha de corte más los días que dura la existencia al ritmo esperado, menos los días de entrega; si ya no da tiempo, el día siguiente al corte.

El esquema se amplió con las columnas nivel_objetivo, cantidad_sugerida y fecha_limite, con su migración para las bases existentes. La ruta GET /alertas las devuelve y ordena el riesgo de desabasto por fecha límite, y la página Alertas muestra la cantidad, la fecha y el total sugerido. Con los datos al 31 de diciembre de 2025, las 279 alertas suman 13,334 unidades sugeridas. Como el esquema core no registra órdenes en camino, la cantidad no descuenta lo ya pedido; el manual de usuario lo indica.

Al repetir la simulación de la mejora 1 con una política que solo pide cuando salta la alerta, el nivel de servicio cayó a 89.4 %. La alerta se optimizó para avisar con el mejor equilibrio entre aciertos y falsas alarmas, por lo que su umbral no lleva colchón; usada como punto de reorden, el pedido llega tarde. La conclusión se incorporó al diseño: la alerta prioriza qué revisar y el nivel objetivo decide cuánto pedir.

### 4.1.15 Mejora 4: explicar cada alerta, rediseño y preparación del despliegue

Cada alerta incluye ahora sus dos o tres factores principales en lenguaje sencillo, en la columna motivos. El módulo ml/explicaciones.py aplica reglas transparentes sobre las mismas cantidades con las que se decide la alerta: cuántos días alcanza la existencia contra los días de entrega, la tendencia frente a las últimas ocho semanas, los eventos de calendario próximos, la temporada alta de la categoría, los días que el producto estuvo agotado y la lentitud del proveedor. Cada regla aporta un factor con un puntaje de gravedad y se conservan los de mayor puntaje. Un ejemplo real: "La existencia alcanza para 5 días y un pedido tarda 7 días; se acerca el fin de año; estuvo agotado 4 de los últimos 28 días: vendió menos de lo que se le pidió".

Se prefirieron reglas a la atribución del modelo, como SHAP, por tres razones: la alerta compara existencia contra demanda, así que su porqué se entiende en esos términos; cada frase se puede verificar contra los datos, y el cálculo es inmediato para todas las alertas sin dependencias nuevas. En el dashboard, la tabla de alertas muestra el motivo principal y, al seleccionar filas, aparece la explicación completa con el pedido sugerido.

El dashboard se rediseñó para dejar de verse como una aplicación genérica: barra lateral marino con el logotipo, encabezado tipo folio con el sello del corte de datos, tipografía IBM Plex Sans alojada en el proyecto, indicadores en franjas y bloques por tema. En la página de Inicio se agregaron una barra de salud del inventario y un mapa de la red de doce sucursales, en el que el tamaño de cada punto representa su ingreso y un anillo rojo marca las tres con más alertas pendientes (Figuras 4.14 a 4.19). Los datos, los indicadores y la API no cambiaron.

Por último, se preparó el despliegue. El equipo eligió Azure for Students, que ofrece crédito para estudiantes sin necesidad de tarjeta de crédito. La revisión de la imagen Docker encontró dos problemas que habrían impedido el despliegue: la imagen no incluía el logotipo y el proceso diario dependía de archivos de datos que no viajan en la imagen. Se corrigieron copiando el logotipo y haciendo que el proceso genere los datos simulados cuando no existen. Además, se agregó un servicio de carga inicial y se midió el pico de memoria del reentrenamiento, cercano a 1.2 GB, para dimensionar la tarea diaria en 2 GB. El plan de recursos, pasos y riesgos quedó documentado en docs/06; su ejecución es el siguiente paso del proyecto.

## 4.2 Análisis de los resultados

### 4.2.1 Resumen de resultados

Todos los componentes de la plataforma se evaluaron contra la verdad de referencia del simulador o contra métodos de referencia. La Tabla 4.18 resume los resultados principales; las secciones siguientes los analizan uno por uno. Las cifras corresponden a la semilla 42 y se pueden reproducir con los comandos del manual técnico.

**Tabla 4.18.** *Resumen de resultados por componente*

| Componente | Resultado | Evidencia |
|---|---|---|
| ETL y calidad | 18 de 18 tipos de defecto detectados; 100 % de precisión; 97.3 % a 100 % de exhaustividad; carga en unos 50 s. | reports/etl/calidad.md |
| Modelo de demanda | WAPE producto-semana de 26.9 % a 27.2 %; mejor método en los 12 casos evaluados. | reports/ml/backtest.md |
| Alertas de desabasto | Regla original: anticipan 54.1 % de los faltantes; regla mejorada: precisión de 25.9 % y F1 de 0.281. | reports/ml/backtest.md |

| Componente | Resultado | Evidencia |
|---|---|---|
| Impacto en dinero | 14 % menos ventas perdidas, 12 % menos inventario, ≈ $346 mil al año. | reports/ml/impacto.md |
| Detección de anomalías | F1 de 0.335; detecta 49.6 % de las fallas de 6 a 8 horas. | reports/ml/anomalias.md |
| Respaldos | 16 MB en 3 s; la restauración de 1,525,992 filas cuadró al 100 % en 12 s. | ops.respaldos |
| Pruebas | 116 pruebas automatizadas aprobadas. | GitHub Actions |

*Nota.* Elaboración propia.

### 4.2.2 Calidad de los datos

El ETL detectó los 18 tipos de defecto inyectados (Tabla 4.19). En 17 de ellos la exhaustividad fue de 100 %: se encontraron todos los registros defectuosos. La única excepción fue el correo electrónico inválido, con 72 de 74 casos (97.3 %); los dos restantes son correos del dominio yahoo.com.mx capturados como yahoo.con.mx. La regla corrige la terminación .con al final del correo, pero esta variante queda en medio del dominio y tiene una sintaxis válida, por lo que pasa la validación. Extender la corrección a esa variante es una mejora sencilla para una siguiente versión. La precisión fue de 100 % en todas las reglas: ninguna marcó como defectuoso un registro sano.

**Tabla 4.19.** *Detección de los defectos inyectados por tipo*

| Defecto | Inyectados | Detectados | Exhaustividad | Precisión |
|---|---|---|---|---|
| sucursal_codigo_sucio | 3,621 | 3,621 | 100.0 % | 100.0 % |
| venta_fecha_formato_alterno | 3,621 | 3,621 | 100.0 % | 100.0 % |
| venta_metodo_pago_faltante | 1,811 | 1,811 | 100.0 % | 100.0 % |
| detalle_duplicado | 1,281 | 1,281 | 100.0 % | 100.0 % |
| cliente_estado_variante | 1,200 | 1,200 | 100.0 % | 100.0 % |
| venta_duplicada | 724 | 724 | 100.0 % | 100.0 % |
| movimiento_cantidad_cero | 648 | 648 | 100.0 % | 100.0 % |
| detalle_cantidad_negativa | 641 | 641 | 100.0 % | 100.0 % |
| detalle_precio_escala | 641 | 641 | 100.0 % | 100.0 % |
| cliente_email_faltante | 640 | 640 | 100.0 % | 100.0 % |
| venta_total_inconsistente | 543 | 543 | 100.0 % | 100.0 % |
| cliente_telefono_faltante | 480 | 480 | 100.0 % | 100.0 % |
| detalle_producto_inexistente | 320 | 320 | 100.0 % | 100.0 % |
| cliente_casi_duplicado | 160 | 160 | 100.0 % | 100.0 % |
| venta_fecha_futura | 89 | 89 | 100.0 % | 100.0 % |
| cliente_email_invalido | 74 | 72 | 97.3 % | 100.0 % |
| producto_categoria_faltante | 4 | 4 | 100.0 % | 100.0 % |
| producto_costo_mayor_precio | 4 | 4 | 100.0 % | 100.0 % |

*Nota.* Elaboración propia con base en reports/etl/calidad.md.

Detectar un defecto no basta; también importa qué tan cerca quedan los datos limpios de los originales. La Tabla 4.20 muestra la carga por tabla. Los registros descartados corresponden a duplicados exactos, renglones con productos inexistentes y los clientes casi duplicados que se fusionaron con su registro original.

**Tabla 4.20.** *Carga de datos por tabla*

| Tabla | Leídas | Cargadas | Descartadas |
|---|---|---|---|
| sucursales | 12 | 12 | 0 |
| categorias | 10 | 10 | 0 |
| proveedores | 35 | 35 | 0 |
| productos | 400 | 400 | 0 |
| clientes | 8,160 | 8,000 | 160 |
| ventas | 181,778 | 181,045 | 733 |
| detalle_venta | 641,954 | 640,353 | 1,601 |
| movimientos_inventario | 647,613 | 647,613 | 0 |
| inventario | 4,800 | 4,800 | 0 |
| incidencias | 6,720 | 6,720 | 0 |

*Nota.* Elaboración propia con base en reports/etl/calidad.md.

Al comparar los datos limpios con la referencia, los productos, las cantidades, los precios y los subtotales del detalle de venta coinciden al 100 %, igual que la sucursal y el cliente de cada venta y las cantidades de los movimientos. El total de la venta coincide en 99.8 %. Las diferencias restantes son esperadas: 9 ventas y 320 renglones no llegaron a la tabla limpia porque sus productos no existen en el catálogo, y 8.2 % de los correos y 5.9 % de los teléfonos de clientes no se pueden recuperar porque venían vacíos en el origen. En esos casos el ETL los marca en lugar de inventar un valor.

### 4.2.3 Pronóstico de demanda

La Figura 4.21 compara el WAPE por producto y semana del modelo contra los cuatro métodos de referencia en los tres horizontes evaluados, con la validación de origen móvil sobre las 52 semanas de 2025.

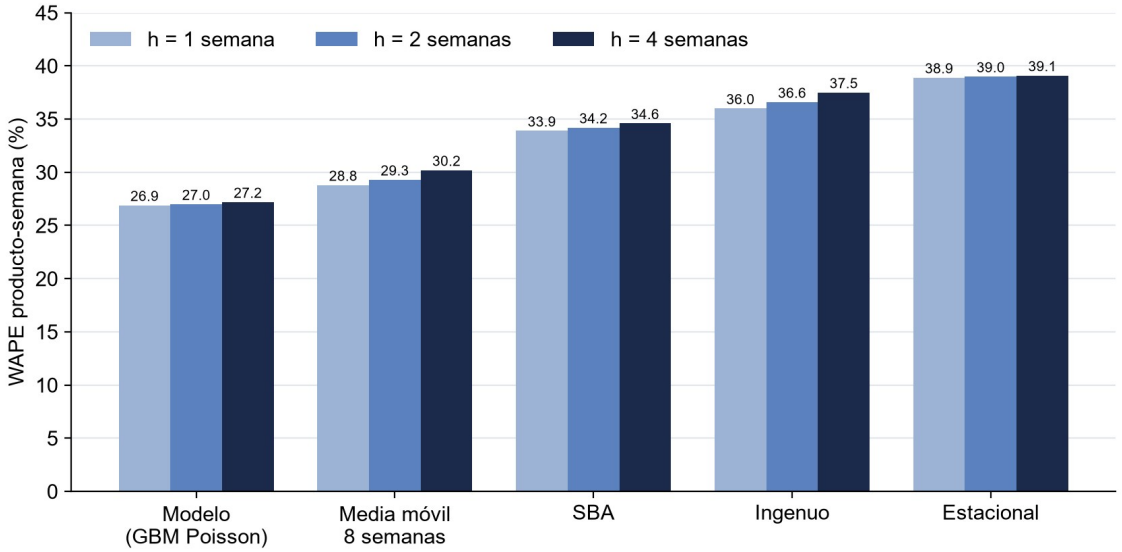



**Figura 4.21.** *WAPE del modelo contra los métodos de referencia*

*Nota.* Elaboración propia con base en reports/ml/backtest.md. Validación con origen móvil: 4 bloques de 13 semanas de 2025.

El modelo fue el mejor método en los 12 casos evaluados: 4 bloques de 13 semanas por 3 horizontes. Su ventaja sobre la media móvil, el mejor de los métodos de referencia, va de 0.6 puntos en el bloque de abril a junio con horizonte de una semana, a 4.4 puntos en el bloque de octubre a diciembre con horizonte de cuatro semanas. La ventaja crece justo en la temporada de Buen Fin y Navidad, cuando el calendario más pesa y los métodos que solo miran el pasado reciente se quedan atrás. Pronosticar a cuatro semanas en lugar de una sube el error del modelo apenas 0.3 puntos, mientras que el de la media móvil sube 1.4 puntos. El sesgo del modelo se mantuvo entre −1.4 % y −1.1 %, es decir, prácticamente sin tendencia a quedarse corto ni a pasarse.

El error depende mucho de la clase ABC del producto (Tabla 4.21). En los productos de clase A, que generan 80 % del ingreso, el WAPE por par sucursal-producto es de 58.7 %; en los de clase C, que venden pocas unidades de forma muy irregular, supera 100 %. Este comportamiento es normal en demanda intermitente: cuando un par vende una o dos unidades al mes, cualquier diferencia es grande en términos relativos. En todas las clases el modelo es el mejor método. La consecuencia práctica es que las alertas deben basarse en intervalos y no solo en el valor puntual.

**Tabla 4.21.** *WAPE por par sucursal-producto y semana según la clase ABC (horizonte de una semana)*

| Método | Clase A | Clase B | Clase C | Todos los pares |
|---|---|---|---|---|
| Modelo (GBM Poisson) | 58.7 % | 82.6 % | 112.6 % | 77.5 % |
| Media móvil de 8 semanas | 61.9 % | 87.1 % | 116.1 % | 81.1 % |
| SBA | 67.4 % | 87.1 % | 119.0 % | 84.4 % |
| Ingenuo | 79.1 % | 108.8 % | 137.1 % | 100.5 % |
| Estacional | 85.1 % | 106.1 % | 129.0 % | 100.9 % |

*Nota.* Elaboración propia con base en reports/ml/backtest.md.

Gracias a que el simulador registra la demanda no surtida, fue posible medir el modelo contra la demanda real y no solo contra las ventas. Contra la demanda real, el sesgo del modelo es de −6.2 % por par: la mayor parte de la demanda perdida por faltantes no se ve en ninguna venta y, por lo tanto, ningún modelo entrenado con ventas puede recuperarla por completo. Se probó la estrategia habitual de excluir del entrenamiento las semanas con tres o más días sin existencia, y el resultado fue contrario a lo esperado: el sesgo contra la demanda real empeoró de −6.4 % a −9.2 % por producto y semana (Tabla 4.22). La explicación más plausible es que las variables de disponibilidad ya le indican al modelo cuándo la venta estuvo limitada, y al quitar esas semanas se pierde justo la información que le permite aprenderlo. Por esta razón, el modelo publicado se entrena con todas las semanas.

**Tabla 4.22.** *Efecto de excluir las semanas censuradas (producto-semana, horizonte de una semana)*

| Variante del modelo | WAPE contra demanda real | Sesgo contra demanda real |
|---|---|---|
| Entrenado con todas las semanas (publicado) | 27.3 % | −6.4 % |
| Excluye semanas con 3 o más días sin existencia | 27.7 % | −9.2 % |

*Nota.* Elaboración propia con base en docs/05_modelo_demanda.md (primer backtest de 13 semanas).

### 4.2.4 Alertas de desabasto (regla original)

Para saber si las alertas anticipan faltantes reales, se aplicó la regla cada semana del periodo de prueba con el pronóstico disponible en ese momento y la existencia al cierre de la semana, y se revisó si en las dos semanas siguientes hubo al menos un día sin existencia. Fueron 60,527 casos con existencia positiva, de los cuales 3,102 (5.1 %) terminaron en faltante. La Tabla 4.23 compara la regla publicada con dos alternativas.

**Tabla 4.23.** *Evaluación de las reglas de alerta de desabasto*

| Regla | Alertas | Aciertos | Precisión | Exhaustividad | F1 |
|---|---|---|---|---|---|
| Modelo, percentil 90 (publicada) | 10,359 | 1,677 | 16.2 % | 54.1 % | 0.249 |
| Modelo, valor medio | 3,436 | 955 | 27.8 % | 30.8 % | 0.292 |
| Promedio de 4 semanas, sin modelo | 5,019 | 1,029 | 20.5 % | 33.2 % | 0.253 |

*Nota.* Elaboración propia con base en docs/05_modelo_demanda.md.

La Figura 4.22 ubica las tres reglas según su precisión y su exhaustividad. Con el mismo umbral, el valor medio, el modelo es más preciso que el promedio simple, 27.8 % contra 20.5 %, y tiene la mejor F1. La regla publicada usa a propósito el percentil 90: anticipa más de la mitad de los faltantes a cambio de unas cinco alertas que no terminan en faltante por cada acierto. Para un almacén es preferible revisar alertas de más que quedarse sin producto, sobre todo porque la base de comparación es baja: sin ninguna regla, solo 5.1 % de los casos terminan en faltante, y entre las alertas publicadas la proporción sube a 16.2 %, más del triple. Con la validación de origen móvil de 52 semanas y 239,271 casos los resultados se sostienen, con 51.6 % de exhaustividad y 15.5 % de precisión.

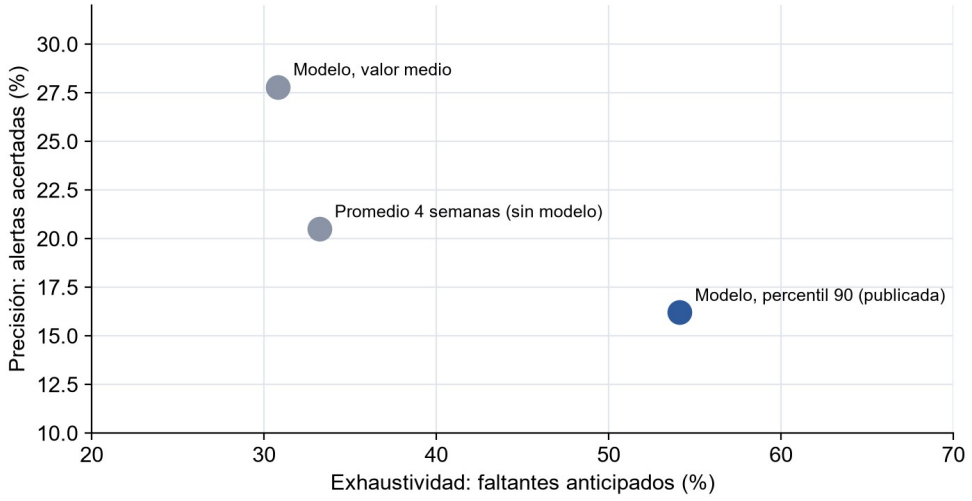



**Figura 4.22.** *Precisión y exhaustividad de las reglas de alerta de desabasto*

*Nota.* Elaboración propia.

### 4.2.5 Detección de anomalías

El detector de anomalías se evaluó contra los 325 días-sucursal en que el punto de venta estuvo caído, información que el simulador registra pero que el detector nunca lee (Tabla 4.24).

**Tabla 4.24.** *Evaluación del detector de anomalías*

| Detector | Umbral | Precisión | Exhaustividad | F1 |
|---|---|---|---|---|
| Poisson sobre tickets (publicado) | z < −2.33 (cola de 1 %) | 38.6 % | 29.5 % | 0.335 |
| Poisson sobre tickets | z < −2.58 (cola de 0.5 %) | 43.2 % | 22.5 % | 0.296 |
| MAD sobre ingreso y tickets (primera versión) | z < −2.5 | 23.8 % | 13.5 % | 0.173 |

*Nota.* Elaboración propia con base en reports/ml/anomalias.md.

El cambio a la prueba de Poisson casi duplicó la F1 de la primera versión, de 0.173 a 0.335. La Figura 4.23 muestra que la capacidad de detección depende de la duración de la falla: el detector encuentra la mitad de las fallas de 6 a 8 horas, pero solo 12.7 % de las de 2 a 3 horas. Una falla corta reduce los tickets del día entre 17 y 25 %, una variación que el ruido normal de un día ya alcanza, por lo que es casi indetectable con datos diarios. El umbral se eligió por F1 sobre el mismo historial, de modo que esta evaluación es exploratoria y no fuera de muestra.

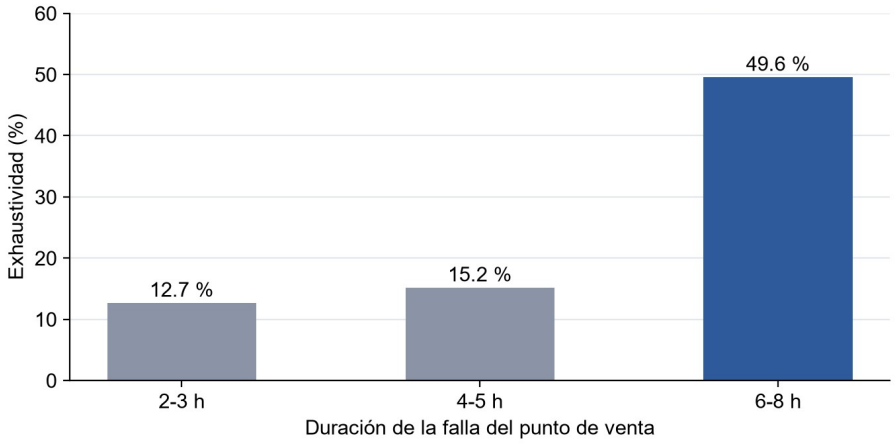



**Figura 4.23.** *Exhaustividad del detector según la duración de la falla*
*Nota.* Elaboración propia con base en reports/ml/anomalias.md.

### 4.2.6 Operación de la plataforma

El respaldo de la base completa pesa 16 MB y se genera en unos 3 segundos. La prueba de recuperación restauró 15 tablas con 1,525,992 filas en una base temporal en unos 12 segundos, y el conteo de filas coincidió al 100 % con la base original. La integración continua ejecuta la cadena completa en cada cambio, desde la generación de datos hasta la construcción y prueba del contenedor, y al cierre de la etapa de mejoras reunía 116 pruebas aprobadas.

### 4.2.7 Cumplimiento de los requerimientos

La Tabla 4.25 resume el estado de cada requerimiento funcional al cierre del proyecto.

**Tabla 4.25.** *Cumplimiento de los requerimientos funcionales*

| ID | Requerimiento | Estado | Evidencia |
|---|---|---|---|
| RF-01 | Dataset simulado reproducible | Cumplido | 181,054 tickets; prueba de reproducibilidad. |
| RF-02 | Ingesta automática de todas las áreas | Cumplido | CSV, Excel y Windows-1252 en el ETL. |
| RF-03 | Limpieza y validación con reglas de calidad | Cumplido | 18 de 18 tipos detectados. |
| RF-04 | Registro de hallazgos y ejecuciones | Cumplido | ops.calidad_hallazgos y ops.etl_ejecuciones. |
| RF-05 | Carga con integridad referencial | Cumplido | Llaves foráneas en core; pruebas de llaves. |
| RF-06 | Indicadores | Cumplido | 9 vistas del esquema kpi. |
| RF-07 | Pronóstico semanal | Cumplido | Mejor método en 12 de 12 casos. |
| RF-08 | Alertas de inventario | Cumplido | 3 tipos de alerta; 54.1 % de faltantes anticipados. |
| RF-09 | API | Cumplido | 27 rutas con clave de acceso. |
| RF-10 | Dashboard con filtros | Cumplido | 6 páginas. |
| RF-11 | Respaldo y restauración | Cumplido | Recuperación verificada al 100 %. |
| RF-12 | Monitoreo y registro de eventos | Cumplido | ops.monitoreo y bitácoras. |
| RF-13 | Detección de anomalías | Cumplido | F1 de 0.335. |

*Nota.* Elaboración propia.

Los requerimientos no funcionales también se cumplieron: la semilla reproduce los mismos datos (RNF-01); el ETL tarda unos 50 segundos frente al límite de 10 minutos (RNF-02); cada hallazgo guarda su tabla y llave de origen (RNF-03); el ETL es idempotente (RNF-04); se reportan precisión y exhaustividad (RNF-05); las credenciales viven en un archivo fuera del control de versiones y la API consulta con un rol de solo lectura (RNF-06); hay 116 pruebas automatizadas y documentación al día (RNF-07), y la plataforma se ejecuta en Windows y en el contenedor Linux de la integración continua (RNF-08).

### 4.2.8 Impacto económico de DATACORE

La Tabla 4.26 compara, sobre la misma demanda real de 2025, la política actual de NexaTech con la reposición guiada por DATACORE al cuantil calibrado en 2024 (sección 4.1.12).

**Tabla 4.26.** *Impacto de DATACORE contra la política actual (2025)*

| Concepto | Política actual | DATACORE (p99) | Diferencia |
|---|---|---|---|
| Ventas perdidas (unidades) | 46,227 | 39,745 | 14.0 % menos |
| Ventas perdidas (pesos) | $ 4.63 M | $ 4.38 M | $ 249,403 recuperados |
| Margen perdido | $ 1.39 M | $ 1.37 M | $ 28,405 recuperados |
| Inventario promedio inmovilizado | $ 10.27 M | $ 9.00 M | 12.4 % menos |
| Costo anual de mantener inventario | $ 2.57 M | $ 2.25 M | $ 317,846 de ahorro |
| Nivel de servicio | 95.37 % | 96.02 % | +0.65 puntos |
| Beneficio neto anual | — | — | $ 346,250 |

*Nota.* Elaboración propia con base en reports/ml/impacto.md. Costo de mantener inventario supuesto en 25 % anual del costo.

Con DATACORE, NexaTech habría perdido 14 % menos ventas manteniendo 12 % menos inventario, con un nivel de servicio ligeramente mayor. El beneficio neto estimado es de unos 346 mil pesos al año. La mayor parte proviene del menor costo de mantener inventario y no del margen recuperado: las unidades que se recuperan son, sobre todo, de productos de bajo precio. Por eso el ahorro en pesos crece menos que el ahorro en unidades.

La Figura 4.24 muestra por qué el resultado no depende de un solo cuantil. Cada punto azul es la política DATACORE con un cuantil distinto: los cuantiles altos compran nivel de servicio con inventario y los bajos hacen lo contrario. La política actual queda arriba y a la derecha de esa curva: pierde más ventas y mantiene más inventario que DATACORE en los cuantiles cercanos al calibrado.

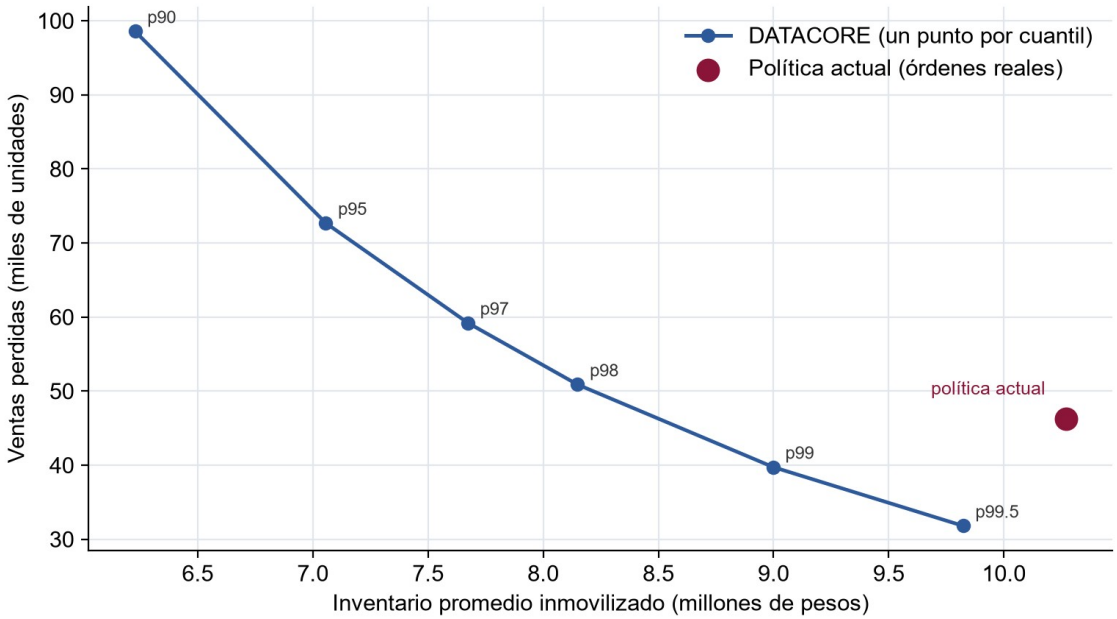



**Figura 4.24.** *Ventas perdidas contra inventario inmovilizado por política, 2025*

*Nota.* Elaboración propia con base en reports/ml/impacto_metricas.csv.

El resultado tiene límites que conviene declarar. Los pronósticos se calcularon con la historia real, la de la política actual, y no con la que habría generado DATACORE; la corrección de censura compensa una parte de esa diferencia. Además, el costo de mantener inventario es un supuesto de la industria y el resultado vale para estos datos simulados.

### 4.2.9 Alertas de desabasto mejoradas

La Tabla 4.27 resume la evaluación de las reglas de alerta en los cuatro trimestres de 2025. Ninguno de ellos participó en la elección de la regla.

**Tabla 4.27.** *Reglas de alerta de desabasto evaluadas fuera de muestra (2025)*

| Regla | Alertas | Aciertos | Precisión | Exhaustividad | F1 |
|---|---|---|---|---|---|
| p90 (publicada hasta la mejora) | 36,803 | 5,728 | 15.6 % | 51.5 % | 0.239 |
| Media | 12,107 | 3,194 | 26.4 % | 28.7 % | 0.275 |
| Promedio de 4 semanas | 18,785 | 3,657 | 19.5 % | 32.9 % | 0.245 |
| Umbral único afinado | 16,096 | 3,762 | 23.4 % | 33.8 % | 0.276 |
| Umbral por segmento | 18,872 | 4,034 | 21.4 % | 36.3 % | 0.269 |
| Media + censura (nueva) | 13,268 | 3,431 | 25.9 % | 30.9 % | 0.281 |

*Nota.* Elaboración propia con base en reports/ml/backtest.md. 239,271 casos y 11,118 faltantes reales.

La regla nueva tiene la mayor F1 y sube la precisión de 15.6 % a 25.9 %: una de cada cuatro alertas termina en faltante real, contra una de cada seis y media con la

regla anterior. A cambio, anticipa menos faltantes (30.9 % contra 51.5 %). Para un comprador que revisa una lista diaria, menos falsas alarmas significa que la lista se atiende completa. La Figura 4.25 muestra que la mejora se sostiene en los cuatro trimestres, mientras que el umbral por segmento queda por debajo de la regla de la media en tres de ellos.

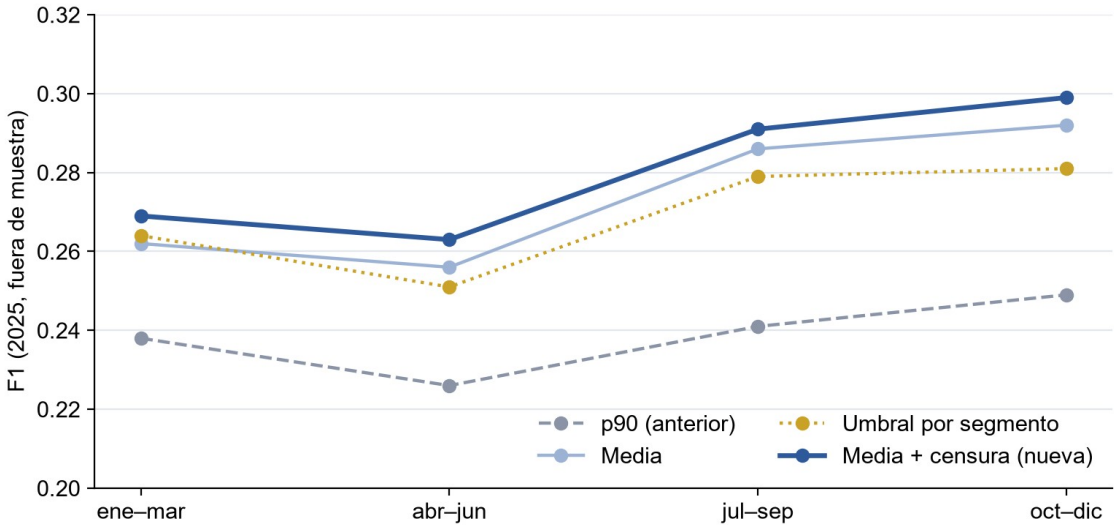



**Figura 4.25.** *F1 por trimestre de 2025 de las reglas de alerta de desabasto*

*Nota.* Elaboración propia con base en reports/ml/backtest.md.

# CAPÍTULO V CONCLUSIONES

## CONCLUSIÓN

## 5.1 Conclusiones generales

DATACORE cumplió su objetivo general: reúne en una sola plataforma reproducible la ingesta, la validación, el almacenamiento, el análisis, el pronóstico, las alertas, la visualización y la operación de los datos de NexaTech Solutions. La plataforma convierte archivos dispersos y defectuosos de cinco áreas en indicadores, pronósticos y alertas que un usuario consulta en el navegador, y lo hace de forma automática cada día.

La decisión de diseño más valiosa fue construir una verdad de referencia desde el simulador. Gracias a ella, cada componente se evaluó con números en lugar de suponer que funcionaba: el ETL contra los defectos inyectados, el modelo contra la demanda real y contra cuatro métodos de referencia, las alertas contra los faltantes que realmente ocurrieron y el detector de anomalías contra las fallas registradas. Esta forma de trabajo permitió además encontrar resultados no evidentes, como que excluir las semanas censuradas empeora el sesgo del modelo, o que las fallas cortas del punto de venta son casi indetectables con datos diarios.

La etapa de mejoras llevó la plataforma de funcionar a demostrar su valor. Una simulación de 2025 sobre la demanda real mostró que reponer con DATACORE habría reducido 14 % las ventas perdidas y 12 % el inventario inmovilizado, con un beneficio neto estimado de 346 mil pesos al año. Las alertas de desabasto pasaron a ser más precisas, a recomendar cuánto y cuándo pedir y a explicar su porqué en lenguaje sencillo.

Los resultados principales confirman el valor de la plataforma. El ETL detecta los 18 tipos de defecto sin falsos positivos y recupera información en lugar de solo descartarla. El modelo de demanda supera a los métodos de referencia en todos los casos evaluados, con más ventaja en la temporada alta, cuando un error cuesta más. La alerta de desabasto, elegida fuera de muestra, acierta en una de cada cuatro ocasiones, contra una de cada seis y media de la regla original, y cada alerta indica cuánto pedir, para cuándo y por qué. La operación incluye respaldos que se restauran y se comparan, monitoreo, bitácoras y pruebas automáticas en cada cambio.

## 5.2 Cumplimiento de los objetivos

La Tabla 5.1 resume el cumplimiento de cada objetivo específico.

**Tabla 5.1.** *Cumplimiento de los objetivos específicos*

| Objetivo específico | Estado | Evidencia |
|---|---|---|
| 1. Arquitectura de software y datos | Cumplido | Arquitectura de cinco capas (Figura 4.2). |
| 2. Base de datos relacional | Cumplido | Esquemas core, ops y kpi en PostgreSQL 16 (Anexo 1). |
| 3. Datos simulados realistas | Cumplido | Tres años, 181,054 tickets, reglas documentadas. |
| 4. Procesos ETL | Cumplido | Carga automática en unos 50 segundos. |
| 5. Reglas de calidad | Cumplido | 18 de 18 tipos detectados, 100 % de precisión. |
| 6. Módulo de aprendizaje automático | Cumplido | WAPE de 26.9 %; alertas con explicación y pedido sugerido; impacto de ≈ $346 mil al año. |
| 7. Dashboard | Cumplido | Seis páginas sobre la API. |
| 8. Respaldos, eventos y monitoreo | Cumplido | Respaldo verificado, bitácoras y monitoreo. |
| 9. Despliegue en un entorno centralizado | Parcial | Contenedor probado en integración continua; plataforma elegida (Azure for Students) y plan documentado; despliegue pendiente. |
| 10. Documentación | Cumplido | Seis documentos técnicos, dos manuales y este reporte. |

*Nota.* Elaboración propia.

## 5.3 Relación con las competencias de la estadía

El proyecto se diseñó para ejercer las tres competencias de la estadía del programa de TSU en Datos e Inteligencia Artificial (Tabla 5.2).

**Tabla 5.2.** *Aplicación de las competencias de la estadía*

| Competencia | Aplicación en el proyecto |
|---|---|
| Analizar los datos de un sistema para su identificación, codificación y tratamiento adecuado de la información. | Diseño de variables, generación de datos, limpieza, reglas de calidad, codificación de llaves, análisis exploratorio e ingeniería de variables del modelo. |
| Manejar arquitecturas de hardware y software para la administración de la información. | Diseño e integración de la base de datos, el ETL, la API, el dashboard, el contenedor y la integración continua. |
| Gestionar el mantenimiento de infraestructura de centros de datos mediante herramientas de hardware y software para asegurar la integridad y calidad de los datos. | Respaldos con prueba de recuperación, bitácoras, monitoreo de servicios, validaciones de integridad, control de errores y pruebas automatizadas. |

*Nota.* Elaboración propia con base en la propuesta del proyecto.

## 5.4 Limitaciones

El proyecto tiene limitaciones que deben considerarse al interpretar sus resultados:

- Los datos son simulados. Aunque las reglas de simulación imitan una operación real, un caso real tendría patrones que el simulador no contempla,

como promociones de la competencia, cambios de precio a mitad de año o errores de captura de otros tipos.

- La validación con origen móvil cubre un solo año, 2025. Con más historia podrían evaluarse más bloques y épocas.
- Las sucursales y los productos nuevos tienen poca historia, y su pronóstico depende más de la categoría, la región y la media del producto.
- La fecha de baja de los productos descontinuados no existe en los datos de negocio, por lo que el modelo los ve como ceros durante el entrenamiento.
- El detector de anomalías casi no ve las fallas cortas con datos diarios y no revisa las fechas especiales del calendario.
- La plataforma no se ha desplegado en un servidor de producción; se ejecuta en el equipo de desarrollo y en la integración continua.
- El impacto en dinero depende de un supuesto de costo de mantener inventario (25 % anual) y de una simulación; es una estimación, no una medición en operación real.

## 5.5 Trabajo futuro

A partir de las limitaciones se proponen las siguientes mejoras:

- Desplegar la plataforma en Azure for Students siguiendo el plan documentado, con la tarea diaria programada y el monitoreo contra las direcciones públicas.
- Descontar de la cantidad sugerida los pedidos en camino, registrando las órdenes de compra en la base.
- Volver a calcular los pronósticos de la simulación con la historia que habría generado DATACORE, no con la de la política actual.
- Registrar la fecha de baja de los productos y modelar su ciclo de vida.
- Detectar anomalías con datos por hora para encontrar fallas cortas del punto de venta.
- Extender la corrección de correos a variantes como .con.mx.

- Probar la plataforma con datos de una empresa real, aprovechando que el ETL ya maneja los formatos y defectos más comunes.

## 5.6 Experiencia del equipo

El proyecto permitió al equipo recorrer el ciclo completo de una plataforma de datos, desde el diseño hasta la operación, y no solo la etapa de modelado que suele dominar los proyectos escolares. El equipo comprobó que la mayor parte del trabajo de un sistema de datos ocurre antes y después del modelo: en entender las fuentes, limpiar los datos, decidir cómo medir y asegurar que todo siga funcionando al día siguiente.

El uso de asistentes de programación con inteligencia artificial aceleró el desarrollo, pero también dejó claro que su trabajo requiere revisión. Los errores más importantes del proyecto, como la elección de la prueba estadística del detector de anomalías o la pérdida del historial de pronósticos al recargar los datos, se encontraron gracias a la revisión humana y a las pruebas automatizadas.

### 5.6.1 Reflexión de David Israel Atilano Quiroz

Para David Israel Atilano Quiroz, DATACORE nació de una decisión. Mientras buscaba vacantes de analista de datos, en lugar de ver la espera como tiempo perdido, decidió usarla para prepararse. Quería que el proyecto sirviera fuera del aula: entender cómo se trabaja con datos en una empresa real y qué podría ofrecerle desde el primer día. Por eso no se quedó en un modelo aislado. Fue la primera vez que construyó un sistema de datos completo, de principio a fin. Empezó simulando tres años de operación de NexaTech con defectos puestos a propósito, como los que llegan de las distintas áreas de un negocio. Después vinieron un ETL que los detecta, un modelo de pronóstico, una API y un dashboard que se respaldan, se monitorean y se prueban de forma automática.

La lección más importante fue que un resultado no vale hasta que se demuestra con datos que el modelo no ha visto. Por eso el trabajo se apoyó en la validación con origen móvil y en comparar siempre contra métodos sencillos. El modelo final dejó el error semanal por producto en 26.9 %, frente a 28.8 % de una media móvil: una mejora modesta, pero real. Esa misma disciplina obligó a descartar ideas que

parecían buenas. Los umbrales de alerta por segmento solo ganaron en uno de cuatro bloques, porque se ajustaban al ruido. Usar la alerta de desabasto como señal de compra habría costado cerca de 1.77 millones de pesos al año, en vez de ahorrar 346 mil. Entender por qué fallaban enseñó tanto como lo que sí funcionó.

Lo más difícil, y también lo más interesante, fue darse cuenta de que las ventas no son la demanda. Cuando un producto se agota, el historial no muestra a los clientes que no lo encontraron, y un modelo entrenado solo con ventas aprende a pedir de menos. Corregir ese sesgo, junto con ajustar el umbral, subió la F1 de las alertas de 0.239 a 0.281. Así, el proyecto pasó de "el modelo predice" a "el sistema le dice a cada sucursal cuánto pedir, para cuándo y por qué". También aprendió a usar asistentes de programación con inteligencia artificial como herramientas que aceleran el trabajo, pero cuyo resultado se revisa y se prueba antes de confiar en él.

Al terminar, la pregunta con la que empezó tiene una respuesta concreta, y el tiempo de espera se convirtió en experiencia. Hoy puede ofrecerle a una empresa más que gráficas: datos limpios y confiables, indicadores que se pueden verificar y recomendaciones traducidas a dinero y a decisiones que cualquier persona del negocio puede entender. Todo se validó con datos simulados, así que su siguiente paso es aplicar esta misma forma de trabajar con datos reales, como analista de datos.

### 5.6.2 Reflexión de Ricardo Leonel Ordaz Aguilar

Durante el desarrollo del proyecto, Ricardo Leonel Ordaz Aguilar comprendió que construir una plataforma de datos no consiste únicamente en desarrollar un modelo, sino que requiere un proceso completo de análisis, preparación, validación y organización de la información. Su participación en la revisión de los avances de cada fase y en la preparación de la defensa le permitió entender mejor las decisiones técnicas que se tomaron, así como la importancia de utilizar métodos adecuados para obtener resultados confiables. También aprendió que técnicas como la pérdida Poisson, pensada para conteos con muchos ceros, y la validación con origen móvil, que evita usar datos del futuro, cumplen una función importante

para mejorar las predicciones y evitar errores en el análisis. Finalmente, el proyecto le permitió fortalecer sus conocimientos en el manejo de datos, desarrollar una visión más completa del trabajo en equipo y comprender cómo las herramientas de análisis pueden utilizarse para resolver problemas reales y apoyar la toma de decisiones.

# REFERENCIAS BIBLIOGRÁFICAS


Batini, C., y Scannapieco, M. (2016). *Data and information quality: Dimensions, principles and techniques*. Springer. https://doi.org/10.1007/978-3-319-24106-7

Codd, E. F. (1970). A relational model of data for large shared data banks. *Communications of the ACM, 13*(6), 377–387. https://doi.org/10.1145/362384.362685

Croston, J. D. (1972). Forecasting and stock control for intermittent demands. *Operational Research Quarterly, 23*(3), 289–303. https://doi.org/10.2307/3007885

Fowler, M. (2006). *Continuous integration*. martinfowler.com. https://martinfowler.com/articles/continuousIntegration.html

Friedman, J. H. (2001). Greedy function approximation: A gradient boosting machine. *The Annals of Statistics, 29*(5), 1189–1232. https://doi.org/10.1214/aos/1013203451

Hyndman, R. J., y Athanasopoulos, G. (2021). *Forecasting: Principles and practice* (3.ª ed.). OTexts. https://otexts.com/fpp3/

Hyndman, R. J., y Koehler, A. B. (2006). Another look at measures of forecast accuracy. *International Journal of Forecasting, 22*(4), 679–688. https://doi.org/10.1016/j.ijforecast.2006.03.001

Kimball, R., y Ross, M. (2013). *The data warehouse toolkit: The definitive guide to dimensional modeling* (3.ª ed.). Wiley.

Kleppmann, M. (2017). *Designing data-intensive applications*. O'Reilly Media.

Merkel, D. (2014). Docker: Lightweight Linux containers for consistent development and deployment. *Linux Journal, 2014*(239), Artículo 2.

Organización de las Naciones Unidas. (2015). *Transformar nuestro mundo: la Agenda 2030 para el Desarrollo Sostenible* (A/RES/70/1). https://undocs.org/es/A/RES/70/1

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., y Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research, 12*, 2825–2830.

Rousseeuw, P. J., y Croux, C. (1993). Alternatives to the median absolute deviation. *Journal of the American Statistical Association, 88*(424), 1273–1283. https://doi.org/10.1080/01621459.1993.10476408

Silver, E. A., Pyke, D. F., y Thomas, D. J. (2017). *Inventory and production management in supply chains* (4.ª ed.). CRC Press.

Syntetos, A. A., y Boylan, J. E. (2005). The accuracy of intermittent demand estimates. *International Journal of Forecasting, 21*(2), 303–314. https://doi.org/10.1016/j.ijforecast.2004.10.001

Tashman, L. J. (2000). Out-of-sample tests of forecasting accuracy: An analysis and review. *International Journal of Forecasting, 16*(4), 437–450. https://doi.org/10.1016/S0169-2070(00)00065-0

Wang, R. Y., y Strong, D. M. (1996). Beyond accuracy: What data quality means to data consumers. *Journal of Management Information Systems, 12*(4), 5–33. https://doi.org/10.1080/07421222.1996.11518099

# ANEXOS

## Anexo 1. Diccionario de datos

Las Tablas A.1 a A.8 describen las tablas del esquema core y la Tabla A.9 las del esquema ops. Las llaves primarias se indican con PK y las foráneas con FK. Los montos están en pesos mexicanos.

**Tabla A.1.** *Diccionario de datos: core.sucursales (12 filas)*

| Columna | Tipo | Nulo | Descripción |
|---|---|---|---|
| id_sucursal | SMALLINT | No | Llave primaria. |
| codigo | VARCHAR(10) | No | Código de negocio único (QRO01, CDMX02…), usado en los archivos crudos. |
| nombre | VARCHAR(80) | No | Nombre comercial. |
| ciudad, estado | VARCHAR | No | Ubicación. |
| region | VARCHAR(20) | No | Bajío, Centro, Occidente o Norte. |
| tamano | VARCHAR(10) | No | grande, mediana o pequena; determina la afluencia. |
| fecha_apertura | DATE | No | Fecha de apertura. |

**Tabla A.2.** *Diccionario de datos: core.categorias (10 filas) y core.proveedores (35 filas)*

| Columna | Tipo | Nulo | Descripción |
|---|---|---|---|
| id_categoria | SMALLINT | No | Llave primaria de categorías. |
| nombre | VARCHAR(40) | No | Nombre de la categoría. |
| id_proveedor | INTEGER | No | Llave primaria de proveedores. |
| nombre | VARCHAR(120) | No | Razón social del proveedor. |
| rfc | VARCHAR(13) | No | RFC de persona moral (ficticio). |
| ciudad, estado, telefono, email | VARCHAR | Sí | Datos de contacto. |
| dias_entrega | SMALLINT | No | Tiempo de entrega comprometido, de 2 a 10 días. |
| activo | BOOLEAN | No | Proveedor vigente. |

**Tabla A.3.** *Diccionario de datos: core.productos (400 filas)*

| Columna | Tipo | Nulo | Descripción |
|---|---|---|---|
| id_producto | INTEGER | No | Llave primaria. |
| sku | VARCHAR(20) | No | Código único NX-<CAT>-<nnnnn>, usado en los archivos crudos. |
| nombre | VARCHAR(120) | No | Artículo, marca ficticia y presentación. |
| id_categoria | SMALLINT | No | FK a categorías. |
| id_proveedor | INTEGER | No | FK a proveedores. |
| precio_unitario | NUMERIC(10,2) | No | Precio de lista vigente (2025). |
| costo_unitario | NUMERIC(10,2) | No | Costo de adquisición; menor que el precio. |
| unidad | VARCHAR(10) | No | Unidad de venta. |
| fecha_alta | DATE | No | Inicio de comercialización. |
| activo | BOOLEAN | No | Falso si el producto se descontinuó. |

**Tabla A.4.** *Diccionario de datos: core.clientes (8,000 filas)*

| Columna | Tipo | Nulo | Descripción |
|---|---|---|---|
| id_cliente | INTEGER | No | Llave primaria. |
| nombre, apellido_paterno, apellido_materno | VARCHAR | No / No / Sí | Nombre completo (ficticio). |
| email | VARCHAR(120) | Sí | Correo electrónico. |
| telefono | VARCHAR(15) | Sí | 10 dígitos con lada de su ciudad. |
| ciudad, estado | VARCHAR | Sí | Domicilio. |
| tipo_cliente | VARCHAR(10) | No | minorista (90 %) o mayorista (10 %). |
| fecha_registro | DATE | No | Alta en el CRM; siempre anterior a su primera compra. |
| id_sucursal_preferida | SMALLINT | Sí | FK a sucursales. |

**Tabla A.5.** *Diccionario de datos: core.ventas (unas 181 mil filas)*

| Columna | Tipo | Nulo | Descripción |
|---|---|---|---|
| id_venta | BIGINT | No | Llave primaria. |
| folio | VARCHAR(20) | No | Código de sucursal y consecutivo; único. |
| fecha_hora | TIMESTAMP | No | Momento del ticket, de 9:00 a 21:00. |
| id_sucursal | SMALLINT | No | FK a sucursales. |
| id_cliente | INTEGER | Sí | FK a clientes; nulo para público general. |
| metodo_pago | VARCHAR(20) | Sí | efectivo, tarjeta_debito, tarjeta_credito o transferencia. |
| total | NUMERIC(12,2) | No | Suma de los subtotales del detalle. |
| estatus | VARCHAR(12) | No | completada o cancelada. |

**Tabla A.6.** *Diccionario de datos: core.detalle_venta (unas 641 mil filas)*

| Columna | Tipo | Nulo | Descripción |
|---|---|---|---|
| id_detalle | BIGINT | No | Llave primaria. |
| id_venta | BIGINT | No | FK a ventas. |
| id_producto | INTEGER | No | FK a productos. |
| cantidad | INTEGER | No | Unidades, mayor que cero. |
| precio_unitario | NUMERIC(10,2) | No | Precio de lista del año de la venta. |
| descuento_pct | NUMERIC(4,3) | No | 0.05 para mayoristas; de 0.10 a 0.25 en Buen Fin para algunas categorías. |
| subtotal | NUMERIC(12,2) | No | cantidad × precio_unitario × (1 − descuento_pct). |

**Tabla A.7.** *Diccionario de datos: core.inventario y core.movimientos_inventario*

| Columna | Tipo | Nulo | Descripción |
|---|---|---|---|
| id_sucursal, id_producto | — | No | Llave primaria compuesta del inventario y FK en los movimientos. |
| existencia | INTEGER | No | Unidades disponibles al cierre del periodo. |
| stock_minimo, stock_maximo | INTEGER | No | Punto de reorden y nivel objetivo vigentes. |
| id_movimiento | BIGINT | No | Llave primaria de los movimientos, en orden cronológico. |
| fecha | DATE | No | Día del movimiento. |

| Columna | Tipo | Nulo | Descripción |
|---|---|---|---|
| tipo | VARCHAR(20) | No | INVENTARIO_INICIAL, ENTRADA_COMPRA, SALIDA_VENTA, MERMA o AJUSTE_CONTEO. |
| cantidad | INTEGER | No | Con signo: positiva si entra, negativa si sale. |
| existencia_resultante | INTEGER | No | Existencia después del movimiento. |
| referencia | VARCHAR(20) | Sí | Orden de compra en las entradas. |

**Tabla A.8.** *Diccionario de datos: core.incidencias (unas 6,700 filas)*

| Columna | Tipo | Nulo | Descripción |
|---|---|---|---|
| id_incidencia | INTEGER | No | Llave primaria. |
| fecha_reporte | DATE | No | Fecha del reporte. |
| id_sucursal | SMALLINT | No | FK a sucursales. |
| id_producto | INTEGER | Sí | FK a productos cuando aplica. |
| tipo | VARCHAR(25) | No | FALLA_SISTEMA, DESABASTO, RETRASO_PROVEEDOR, MERCANCIA_DANADA, DIFERENCIA_INVENTARIO o QUEJA_CLIENTE. |
| severidad | VARCHAR(5) | No | baja, media o alta. |
| descripcion | VARCHAR(250) | No | Texto libre. |
| estatus | VARCHAR(12) | No | abierta, en_proceso o resuelta. |
| fecha_resolucion | DATE | Sí | Solo si está resuelta. |

**Tabla A.9.** *Tablas del esquema ops*

| Tabla | Propósito |
|---|---|
| etl_ejecuciones | Una fila por proceso ETL: duración, filas leídas, cargadas y rechazadas, y error. |
| calidad_hallazgos | Una fila por registro afectado por una regla: regla, acción y valores antes y después. |
| modelos | Versiones de modelos con sus métricas en JSONB y la marca del modelo activo. |
| predicciones_demanda | Pronóstico semanal por sucursal y producto con su intervalo. |
| alertas_inventario | Alertas de desabasto, exceso y sin rotación, con su cobertura, su motivo, su estado de atención y, en el desabasto, el pedido sugerido y la fecha límite. |
| anomalias | Días-sucursal con valor observado, esperado y puntaje de la prueba. |
| respaldos, monitoreo, corridas | Evidencia de la operación: respaldos verificados, chequeos y pasos del proceso diario. |

## Anexo 2. Guía rápida de operación

La Tabla A.10 resume los comandos para instalar y operar la plataforma desde cero. El detalle completo se encuentra en el manual técnico del repositorio.

**Tabla A.10.** *Comandos principales de la plataforma*

| Tarea | Comando |
|---|---|
| Generar los datos simulados | python -m datacore.generator --salida data |
| Crear la base, los roles, las tablas y las vistas | python -m datacore.db_setup |

| Tarea | Comando |
|---|---|
| Ejecutar el ETL con evaluación | python -m datacore.etl --evaluar |
| Evaluar el modelo contra los métodos de referencia | python -m datacore.ml evaluar |
| Entrenar, pronosticar y generar alertas | python -m datacore.ml publicar |
| Simular el impacto en dinero | python -m datacore.ml impacto |
| Comparar las reglas de alerta fuera de muestra | python -m datacore.ml alertas |
| Levantar la API | python -m uvicorn datacore.api.app:app --port 8000 |
| Levantar el dashboard | python -m streamlit run src/datacore/dashboard/Inicio.py |
| Respaldar y verificar | python -m datacore.operacion respaldo --verificar |
| Revisar el estado de la plataforma | python -m datacore.operacion monitoreo |
| Ejecutar el proceso diario | python -m datacore.operacion diario |
| Cargar todo en un servidor nuevo | python -m datacore.operacion inicial |
| Ejecutar las pruebas | python -m pytest -q |

*Nota.* Elaboración propia con base en docs/manual_tecnico.md.

## Anexo 3. Guía rápida para el usuario del dashboard

El dashboard se abre en el navegador, en la dirección que proporcione el área de sistemas. Los datos se actualizan una vez al día, de madrugada, y la fecha de corte aparece en el sello «Corte de datos» del encabezado de cada página. Los montos están en pesos mexicanos. Las tablas se pueden ordenar con un clic en el encabezado y descargar en formato CSV. La Tabla A.11 explica cómo interpretar cada tipo de alerta.

**Tabla A.11.** *Interpretación de las alertas para el usuario*

| Alerta | Qué significa | Qué hacer |
|---|---|---|
| RIESGO_DESABASTO | La existencia probablemente no alcanza hasta que llegue un nuevo pedido. | Revisar el motivo y pedir la cantidad sugerida antes de la fecha límite. |
| EXCESO_INVENTARIO | Al ritmo pronosticado, la existencia dura más de 180 días. | Frenar las compras; considerar promociones o traspasos. |
| SIN_ROTACION | Hay piezas, no se vende nada desde hace 13 semanas y no se espera demanda. | Evaluar liquidar o descontinuar. |

*Nota.* Elaboración propia con base en docs/manual_usuario.md.

Las alertas de desabasto aparecen ordenadas por fecha límite, con su cantidad sugerida y su motivo; al seleccionar filas se muestra la explicación completa. Antes de comprar conviene descontar los pedidos en camino, que la base no registra. Para marcar una alerta como atendida, se selecciona y se pulsa Marcar como atendidas.